

DOSSIER DE PROJET

Titre professionnel Développeur Web et Web Mobile

>_ monPremierPoste_

MON PREMIER POSTE

L'agrégateur d'offres qui met les juniors en avant

Présenté par Mohamed Sadoune

La Plateforme_ • Toulon • Promotion 2025/2026

>_ Sommaire

>_ PARTIE 1 • LE POINT DE DÉPART	4
>_ 1. Prologue.....	4
>_ 2. Le besoin et le cahier des charges.....	5
2.1 Contexte et problématique	5
2.2 La réponse apportée.....	5
2.3 Publics et besoins	5
2.4 Acteurs et rôles	6
2.5 Contraintes	6
2.6 Les trois sources et leurs licences	7
>_ 3. Gestion de projet.....	8
3.1 Méthode et découpage en étapes.....	8
3.2 Branches, pull requests et conventions.....	10
>_ PARTIE 2 • CONCEPTION	12
>_ 4. Conception fonctionnelle.....	12
4.1 User stories.....	12
4.2 Cas d'utilisation.....	12
>_ 5. Conception de la base de données.....	14
5.1 Pourquoi Merise	14
5.2 Du MCD au MLD, puis au MPD.....	14
5.3 Ce que la méthode a apporté	15
>_ 6. Conception des interfaces.....	16
6.1 Charte graphique.....	16
6.2 Wireframes	16
6.3 Maquettes	17
6.4 Diagramme de navigation	17
>_ PARTIE 3 • RÉALISATION	18
>_ 7. Architecture et environnement de travail.....	18
7.1 Schéma d'architecture.....	18
7.2 Le MVC et le rôle des couches.....	18
7.3 Environnement de développement.....	18
7.4 Qualité de code	19
7.5 Choix techniques justifiés	19
>_ 8. Implémentation du back.....	20
8.1 Accès aux données.....	20
8.2 Trois sources, un seul format	21
8.3 Le filtrage des annonces	22
8.4 Tâches planifiées et garde-fous	24
>_ 9. Implémentation du front.....	27
9.1 Interfaces statiques	27
9.2 Partie dynamique de l'interface.....	27
9.3 Accessibilité mise en œuvre	28
9.4 Référencement.....	29
>_ PARTIE 4 • MISE À L'ÉPREUVE	31

>_ 10. Sécurité de l'application.....	31
10.1 Mots de passe et sessions.....	31
10.2 Protection CSRF.....	33
10.3 Injection SQL.....	34
10.4 XSS et échappement des sorties.....	34
10.5 Contrôle d'accès et cloisonnement des documents.....	35
10.6 Rate limiting.....	36
10.7 En-têtes de sécurité.....	37
10.8 RGPD.....	38
>_ 11. Tests et jeu d'essai.....	39
11.1 Stratégie de tests.....	39
11.2 Tests unitaires.....	39
11.3 Tests d'intégration.....	40
11.4 Tests de sécurité.....	40
11.5 Jeu d'essai.....	41
>_ PARTIE 5 • MISE EN PRODUCTION.....	42
>_ 12. Déploiement et exploitation.....	42
12.1 Environnements.....	42
12.2 Comptes et droits d'accès à la base.....	42
12.3 Mise en production : VPS, Docker Compose et Caddy.....	42
12.4 Intégration et déploiement continu avec GitHub Actions.....	43
12.5 Sauvegarde et restauration.....	45
12.6 Supervision et journalisation.....	46
>_ 13. Documentation et veille.....	47
13.1 Documentation technique.....	47
13.2 Veille technologique et de sécurité.....	47
>_ PARTIE 6 • RETOUR SUR LE PROJET.....	49
>_ 14. Bilan et perspectives d'évolution.....	49
>_ 15. Épilogue.....	50
>_ ANNEXES.....	51

>_ PARTIE 1 • LE POINT DE DÉPART

>_ 1. Prologue

Je m'appelle Mohamed Sadoune. Avant d'écrire la première ligne de ce projet, j'ai passé vingt-quatre ans dans le commerce, dont les dernières comme responsable de magasin dans le prêt-à-porter.

C'est un métier de terrain, où l'on apprend à écouter un besoin, à décider vite, et à rendre des comptes sur des résultats. Rien de tout cela ne s'est perdu en chemin.

De formation commerciale, j'ai pourtant croisé l'informatique une première fois. En 2008, j'intègre un BTS Informatique de gestion, option administrateur de réseaux. Le programme comportait une part de développement, modeste : quelques semaines de HTML, de CSS et de SQL.

Ce fut la partie que j'ai préférée, et de loin. J'y ai éprouvé quelque chose que je n'avais pas anticipé : le plaisir de construire une chose qui fonctionne, et de comprendre pourquoi elle fonctionne. Je me souviens m'être dit, à l'époque déjà, que je m'étais peut-être trompé d'orientation, et que le développement web m'aurait mieux convenu.

Je suis retourné au commerce. Seize ans. L'idée, elle, n'est jamais tout à fait partie ; elle est restée dans un coin de la tête, de celles qu'on remet à plus tard sans jamais les abandonner. Cette formation est le moment où je cesse de la remettre.

Reste à choisir ce qu'on construit. J'aurais pu prendre un sujet confortable. J'ai préféré partir d'un problème que je connais parce que je le vis : décrocher un premier poste de développeur est aujourd'hui très difficile.

Les offres qui s'adressent aux juniors sont noyées parmi les autres, et la plupart réclament trois à cinq ans d'expérience minimum, pour des postes présentés comme ouverts aux débutants. Le paradoxe est connu de tous ceux qui cherchent, et il décourage avant même la première candidature.

De là est né **MonPremierPoste** : un agrégateur d'offres pensé pour les développeurs, et surtout doté d'un **filtre junior** que je n'ai trouvé sur aucune des plateformes que j'ai utilisées pendant ma propre recherche.

Le site rassemble les annonces de plusieurs sources publiques, les normalise, écarte celles qui n'ont rien à faire là, et met en avant celles qui sont réellement accessibles à quelqu'un qui commence. Ce n'est pas un exercice d'école déguisé : c'est l'outil que j'aurais voulu avoir.

Ce dossier raconte ce chemin. Il expose le besoin, les choix de conception, la réalisation, et surtout ce que j'ai dû défaire pour le refaire correctement, car c'est là, plus que dans les fonctionnalités réussies, que j'ai appris le métier.

>_ 2. Le besoin et le cahier des charges

2.1 Contexte et problématique

Un développeur web qui cherche son premier poste se heurte à un obstacle que peu de métiers connaissent sous cette forme : il est écarté avant d'avoir pu se montrer.

Les offres réellement accessibles aux débutants (celles qui acceptent un profil peu expérimenté, une alternance ou un stage) sont noyées parmi des milliers d'annonces réclamant trois à cinq ans d'expérience. Elles sont en outre dispersées sur des plateformes qui ne se parlent pas.

J'ai passé vingt-quatre ans dans le commerce, dont plusieurs à recruter. La comparaison éclaire le problème. Dans la vente, un candidat sans expérience est embauché puis évalué : c'est la période d'essai qui tranche, sur le terrain.

En développement, le tri se fait en amont, sur le papier, à partir d'un critère écrit : le nombre d'années. Le candidat junior n'est pas écarté parce qu'il a échoué : il l'est parce qu'il n'a jamais été vu.

Ce déplacement du filtre change la nature du problème. Il ne s'agit pas de rendre le débutant meilleur, mais de le rendre visible, et de lui faire gagner le temps qu'il perd aujourd'hui à trier manuellement des annonces qui ne lui sont pas destinées.

S'ajoute une difficulté d'organisation : celui qui candidate sur cinq plateformes ne dispose d'aucun outil pour savoir où il en est. Les relances s'oublient, les réponses se perdent, et le découragement s'installe avant le premier entretien.

2.2 La réponse apportée

MonPremierPoste répond à ces deux difficultés par un point d'entrée unique, orienté premier emploi, qui va de la découverte de l'offre jusqu'au suivi de la candidature.

Agréger ce qui est dispersé. Les offres de plusieurs sources publiques sont récupérées, normalisées vers un format commun et dédoublonnées. L'utilisateur consulte un catalogue unique au lieu de visiter chaque plateforme.

Trier pour les juniors. Chaque offre reçoit un indicateur d'accessibilité aux débutants, calculé à l'ingestion. Le filtre junior est actif par défaut : le bruit des annonces inaccessibles est écarté sans que l'utilisateur ait à le demander, et il peut à tout moment afficher l'ensemble du catalogue.

Suivre jusqu'au bout. La plupart des plateformes s'arrêtent à la consultation. Ici, la recherche se pilote de bout en bout : postuler, relancer si besoin, et faire évoluer le statut de chaque candidature jusqu'à son issue.

S'adresser au candidat, pas au marché. Le nom du site est écrit à la première personne (*mon* premier poste), et ce parti pris a des conséquences concrètes : le filtre junior est actif sans qu'on le demande, les messages expliquent au lieu de constater, et le suivi rend visible une progression là où le candidat n'a d'ordinaire aucun retour. Un annuaire recense ; ce site accompagne.

2.3 Publics et besoins

Le public cible. Les développeurs web en recherche d'un premier emploi, d'une alternance ou d'un stage.

- Sortants de formation, dont les candidats au titre professionnel.
- Personnes en reconversion, qui cumulent l'absence d'expérience dans le métier et un parcours antérieur à expliquer.

- Juniors déjà en poste qui souhaitent changer.

Leur besoin commun n'est pas d'avoir plus d'offres, mais d'en avoir moins et de meilleures : un catalogue où chaque annonce affichée leur est réellement ouverte.

Le filtre junior reste toutefois un réglage, pas une restriction d'accès : désactivé, il rend l'intégralité du catalogue. Le site sert donc aussi les développeurs confirmés, et un profil qui gagne en expérience n'a pas à changer d'outil.

2.4 Acteurs et rôles

Trois rôles se partagent l'application, selon un principe de droits croissants.

Rôle	Ce qu'il peut faire
Visiteur	Consulter et filtrer les offres, ouvrir une fiche, lire les pages légales. Il ne peut ni postuler ni suivre.
Candidat	Tout ce que fait le visiteur, plus : gérer son profil et ses documents, mettre en favori, postuler, relancer, suivre ses candidatures, exercer ses droits sur ses données.
Administrateur	Superviser : tableau de bord, gestion des comptes, activation des sources et synchronisation manuelle, modération des offres.

Une limite est posée volontairement. L'administrateur ne consulte ni le contenu des candidatures, ni les documents déposés par les utilisateurs : il ne voit que des compteurs agrégés. Cette restriction relève de la minimisation des données, et elle est inscrite dans le code, pas seulement dans une intention.

2.5 Contraintes

Sécurité. Le service conserve des données que ses utilisateurs ne confient pas à la légère : leur adresse électronique, leur CV, leur lettre de motivation, et de quoi vérifier leur mot de passe sans jamais le détenir. Il ouvre par ailleurs des formulaires à tout visiteur et un espace réservé à l'administration.

Deux exigences en découlent, posées avant la première ligne de code. Ce qui appartient à un candidat n'est accessible qu'à lui. Et rien de ce qui vient du navigateur n'est tenu pour sûr, quelle que soit l'apparence de la requête. Les moyens employés pour les tenir occupent la section 10 tout entière.

Protection des données. Les documents déposés sont des données personnelles : consentement recueilli à l'inscription, conservation liée à la vie du compte, droit d'accès et droit à l'effacement mis en œuvre. Les adresses de recruteurs issues des sources font l'objet d'un traitement prudent, imposé par la licence de l'un des fournisseurs.

Accessibilité. Libellés explicites, contrastes vérifiés, navigation au clavier, régions vivantes annoncées aux lecteurs d'écran.

Affichage. Le site s'adapte du grand écran au téléphone, par une série de breakpoints successifs décrits en 9.1.

Performance et sobriété. Les offres sont lues en base, jamais redemandées aux interfaces distantes à chaque visite : l'affichage reste rapide, les quotas des fournisseurs sont préservés, et le nombre d'échanges réseau reste faible. Les réponses sont compressées et les fichiers statiques mis en cache.

Robustesse. Codes de réponse cohérents, pages d'erreur dédiées, gestion des erreurs centralisée.

2.6 Les trois sources et leurs licences

Le cahier des charges envisageait sept sources. Trois ont été retenues.

Source	Couverture	Ce que sa licence impose
France Travail	France, gros volume, officiel	Marque et logo interdits à la reproduction. Mention de la licence et date de mise à jour obligatoires. Anonymisation des offres retirées.
La Bonne Alternance	France, alternance, officiel	Licence Ouverte 2.0. Paternité à mentionner, avec la date de dernière mise à jour.
Adzuna	France et international, agrégateur	Attribution visible imposée sous forme de badge, et quotas d'appels contractuels.

Les quatre autres ont été écartées, pour deux raisons distinctes. Arbeitnow, Remotive et Remote OK sont internationales et centrées sur le télétravail mondial. Elles auraient apporté des offres, y compris ouvertes aux débutants, mais hors du périmètre que le projet s'est fixé : le premier poste en France. Les retenir supposait d'ouvrir le catalogue à l'international et de donner à l'utilisateur de quoi s'y retrouver. C'est une évolution identifiée, décrite en 15, plutôt qu'un renoncement. La Bonne Boîte, elle, ne diffuse pas d'offres : elle recense des entreprises susceptibles de recruter, une donnée sans intitulé de poste ni date de publication, donc sans rien à normaliser ni à suivre dans une candidature.

Les trois licences retenues sont contradictoires entre elles. Toutes trois sont gratuites et publiques, et pourtant l'une interdit la reproduction de son logo quand une autre l'impose. Aucune règle d'affichage unique n'était donc possible : chaque source déclare ses propres obligations dans sa configuration, et l'interface s'y conforme source par source, un mécanisme décrit en 8.2.

>_ 3. Gestion de projet

Le développement n'a pas commencé par du code, mais par un plan. Avant la première ligne, j'ai rédigé un plan de développement qui découpe le projet en onze étapes, nomme les branches Git de chacune (**feat/<nom>-back** puis **feat/<nom>-front**) et énumère les tâches à accomplir, une par une, dans l'ordre où elles doivent être traitées.

Ce plan a ensuite été transcrit dans GitHub, où il est devenu l'outil de suivi quotidien : une étape est devenue un **milestone**, une tâche une **issue** numérotée, rattachée à son milestone et classée par un **label** : CONCEPTION, BACK, FRONT ou TEST.

Travailler seul supprime la revue par les pairs. Aucun collègue ne relit une pull request, ne s'étonne d'un nom de variable, ne remarque qu'une vérification est écrite à deux endroits. Ce garde-fou manquant a été remplacé par des règles posées à l'avance et tenues sans exception : conventions de nommage des branches et des commits, auto-revue du diff dans la pull request avant chaque merge, et critères d'acceptation fixés dans l'issue avant d'écrire la moindre ligne.

Trois règles ont structuré tout le projet : un milestone ne s'ouvre pas tant que le précédent n'est pas mergé sur main, le back se développe avant le front, et aucune branche ne rejoint main sans pull request relue. Les trois sont détaillées ci-dessous, et l'historique du dépôt permet de vérifier qu'elles ont bien été suivies.

3.1 Méthode et découpage en étapes

Onze étapes, une par lot fonctionnel. Le projet est découpé en onze étapes, numérotées de 0 à 10. Chacune regroupe les tâches d'un même lot (l'authentification, les offres, les favoris) et non celles d'une même couche technique. Une étape va donc de la table à l'écran, et se termine sur quelque chose qui se lance et se teste.

Trois étapes font exception, et c'est assumé. L'étape 0 précède le code : c'est la conception, livrée sous forme de documents. Les étapes 4, l'agrégation des offres, et 10, tests, conteneurisation et déploiement, sont purement serveur et n'ont pas de contrepartie à l'écran.

Milestone	Objet	Issues	Mergée le
0	Conception : cahier des charges, user stories, cas d'utilisation, MCD, MLD, MPD, charte, wireframes, maquettes	11	avant le dépôt
1	Initialisation : squelette MVC, base de données, templates communs	15	1er juillet
2	Authentification et comptes : inscription, connexion, sessions, mot de passe oublié	16	6 et 7 juillet
3	Profil et RGPD : documents, consultation et anonymisation des données	10	9 juillet
4	Agrégation : connecteurs, normalisation, dédoublonnage, tâche planifiée	14	12 juillet
5	Offres : recherche, filtres en direct, page de détail	8	12 et 15 juillet
6	Favoris : ajout, retrait, page dédiée	4	16 et 17 juillet
7	Candidatures : envoi par courriel, suivi, relance	9	17 et 20 juillet
8	Administration : tableau de bord, comptes, sources, modération	10	20 et 23 juillet

9	Légal et accessibilité : pages légales, pages d'erreur, durcissement	7	27 et 28 juillet
10	Tests et déploiement : jeu d'essai, tests, conteneurisation, intégration continue, mise en ligne	9	28 juillet au 26 août
	Total des issues rattachées à un milestone	113	

Les dates se lisent par paires : la branche back d'abord, la branche front ensuite. Onze semaines séparent le premier commit du dernier, et le rythme change en route : trois semaines pour la seule étape 1, puis huit étapes en six semaines. C'est l'effet attendu d'un socle : une fois la structure et les conventions en place, chaque fonctionnalité réutilise l'existant au lieu de l'inventer.

Chaque tâche est une issue. Le plan comptait cent deux tâches numérotées ; chacune est devenue une issue portant son numéro dans son titre. L'étape 6, « Favoris », donne la mesure du grain retenu, avec quatre issues, dans l'ordre où elles ont été traitées :

- 64 · models/Favori.js · BACK
- 65 · Controller favoris · BACK
- 66 · Bouton favori (fetch) · FRONT
- 67 · Page Mes favoris + filtres · FRONT

Une issue tient donc dans un fichier, ou dans un écran. C'est le grain qui permet de fermer quelque chose chaque jour, et de savoir à tout moment où l'on en est.

Les quatre labels se répartissent très inégalement. BACK 61, FRONT 33, CONCEPTION 11, TEST 2. Ce déséquilibre n'est pas un défaut de classement : il donne le poids réel de chaque côté du projet, et il explique pourquoi le serveur a occupé l'essentiel du temps. Filtrer le board sur un label donne par ailleurs ce qui reste à faire de cette nature, tous milestones confondus.

Un board kanban en cinq colonnes. Les issues sont disposées sur un tableau GitHub Projects, en vue Board. Chaque colonne porte sa définition, écrite dans le tableau lui-même :

Colonne	Ce qu'elle contient
Backlog	Tâches créées, pas encore planifiées.
Ready	Sélectionnée pour l'étape en cours, prête à développer.
En cours	En cours de développement.
En revue	Pull request ouverte, auto-revue avant le merge.
Terminé	Mergé dans main.

Une carte n'avance donc jamais sur une impression, mais sur un fait vérifiable : elle passe en « En revue » à l'ouverture de la pull request, et en « Terminé » au merge.

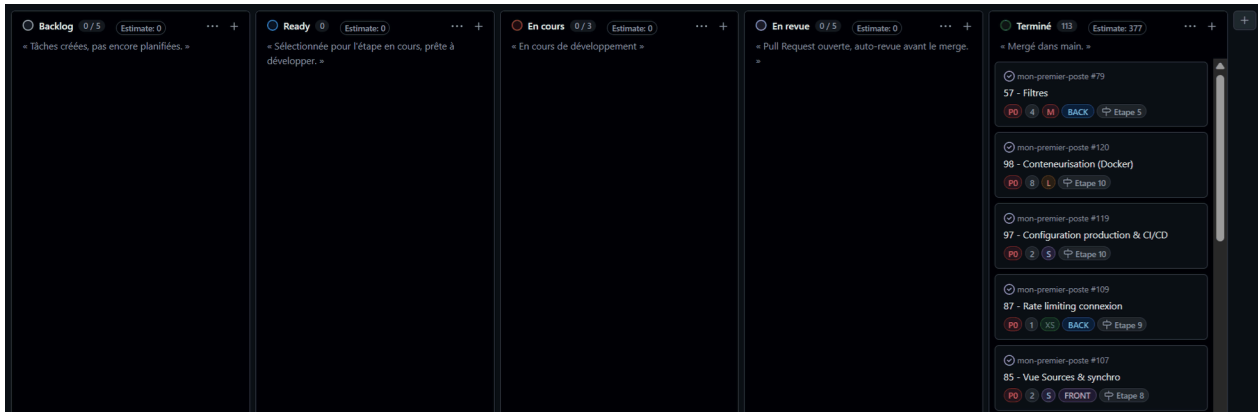


Figure 1. Le board GitHub Projects en fin de projet. Chaque colonne porte sa définition et sa limite de cartes : trois en cours au maximum, cinq en revue. Les cartes affichent leur numéro d'issue, leur numéro de tâche, leur taille, leur label et leur étape. Les cent treize sont dans « Terminé », les colonnes de travail sont vides : le projet est clos.

3.2 Branches, pull requests et conventions

Deux branches par étape. Une branche back, mergée sur main dès qu'elle est terminée, puis une branche front créée depuis un main déjà à jour. Ce n'est pas un détail d'organisation : c'est ce qui garantit que le front est développé contre le back réellement intégré, et non contre une version locale qui n'a pas encore été confrontée au reste.

Une pull request par branche, quarante-deux en tout, chacune intégrée par un merge commit. Relire son propre travail dans le diff n'a rien d'un rituel : le diff présente le code hors de son contexte d'écriture, et fait apparaître ce qu'on ne voit plus dans l'éditeur : une variable laissée là, un fichier de débogage oublié, un commentaire devenu faux.

Des messages de commit normalisés. Le dépôt compte 190 commits, dont 42 de fusion. Les 148 autres suivent la convention des commits typés : un préfixe qui annonce la nature du changement, un verbe à l'impératif, et le numéro de l'issue entre parenthèses.

Type	Nombre	Ce qu'il désigne
feat	106	Une fonctionnalité nouvelle.
fix	15	Une correction de comportement.
docs	9	Documentation, commentaires, README.
chore	6	Maintenance sans effet fonctionnel.
build	3	Empaquetage et conteneurisation.
test	2	Ajout ou modification de tests.
refactor	2	Réécriture sans changement de comportement.
ci	2	Chaîne d'intégration continue.
style	1	Mise en forme, sans effet sur la logique.
perf	1	Amélioration de performance.

Ces préfixes servent à chercher. Quand un comportement inattendu apparaît, ils permettent de filtrer l'historique au lieu de le parcourir : un **feat** a ajouté quelque chose, un **fix** a corrigé quelque chose, et un

refactor affirme n'avoir rien changé au comportement. Trois pistes qui orientent la recherche plutôt que de la faire commencer au hasard.

Le numéro d'issue entre parenthèses complète le dispositif : d'un commit on remonte à l'issue qui l'a motivé, donc aux critères d'acceptation qui disaient ce que le code devait faire.

Le nom des branches suit la même logique que les commits : le préfixe annonce la nature du travail.

☐	0 Open	42 Closed	Author	Label	Projects	Milestones	Reviews	Assignee	Sort
☐				Étape 8 (front) — Administration : navigation, dashboard, utilisateurs, sources, modération					
			#138 by mohamed-sadoune83 was merged on Jul 23						
☐				Étape 8 (back) — Administration : dashboard, utilisateurs, sources, modération					1
			#137 by mohamed-sadoune83 was merged on Jul 20						
☐				Étape 7 (front) — Candidatures : postuler, suivi, actions, filtres favoris					
			#136 by mohamed-sadoune83 was merged on Jul 20						
☐				Étape 7 back — Candidatures : envoi par email, suivi et relance					
			#135 by mohamed-sadoune83 was merged on Jul 17						
☐				Étape 6 front — Favoris : bouton cœur et page « Mes favoris »					
			#134 by mohamed-sadoune83 was merged on Jul 17						
☐				Étape 6 back — Favoris : modèle et endpoints d'ajout/retrait					
			#133 by mohamed-sadoune83 was merged on Jul 16						
☐				feat: écarte les offres hors-cible « développeur » à l'ingestion					
			#132 by mohamed-sadoune83 was merged on Jul 15						
☐				fix: écarte les postes seniors de la détection « junior » (veto sur le titre)					
			#131 by mohamed-sadoune83 was merged on Jul 15						
☐				Étape 5 — Offres (front) : liste, fetch et page détail					1
			#130 by mohamed-sadoune83 was merged on Jul 15						
☐				Ajout du type de contrat intérim					
			#129 by mohamed-sadoune83 was merged on Jul 12						
☐				Étape 5 (back) — Liste, filtres, recherche et détail des offres (#56–#60)					
			#128 by mohamed-sadoune83 was merged on Jul 12						
☐				Étape 4 — Agrégation des offres (#42–#55)					
			#127 by mohamed-sadoune83 was merged on Jul 12						
☐				Merge Étape 3 — Profil, documents & RGPD (front) (#126)					1
			#126 by mohamed-sadoune83 was merged on Jul 9						
☐				Merge Étape 3 — Profil, documents & RGPD (back) (#125)					1
			#125 by mohamed-sadoune83 was merged on Jul 9						

Figure 2. Les quarante-deux pull requests du dépôt, toutes fermées, de la plus récente à la plus ancienne. Les étapes à deux volets y occupent deux lignes, la back puis la front ; l'étape 4, purement serveur, n'en a qu'une.

>_ PARTIE 2 • CONCEPTION

>_ 4. Conception fonctionnelle

Entre le cahier des charges et le modèle de données, il manque une marche. Le cahier des charges dit ce que le site doit permettre ; le modèle de données dit comment l'information sera rangée. Entre les deux, il faut décrire ce que chaque utilisateur fait réellement, et à quelle condition on considérera que c'est fait.

C'est l'objet de cette section. Elle correspond aux deux premières issues de l'étape 0 après le cahier des charges : les user stories, puis le diagramme de cas d'utilisation qui les met en scène.

4.1 User stories

Une user story dit un besoin, pas une solution. Elle décrit ce qu'un utilisateur veut obtenir et pourquoi, avant qu'on ait décidé comment le faire. La formule *en tant que [acteur], je veux [action], afin de [bénéfice]* garde le « pourquoi » attaché au « quoi » : une fonctionnalité dont on n'arrive pas à énoncer le bénéfice est une fonctionnalité qu'on est en train de s'inventer.

Un exemple, tiré du backlog. C'est la première user story de l'épopée Favoris, la fonctionnalité qui sert de fil conducteur à ce dossier, et qu'on retrouvera en base de données, dans le modèle et à l'écran.

US17	Favoris
En tant que	Candidat
Je veux	ajouter ou retirer une offre de mes favoris
Afin de	garder de côté les offres qui m'intéressent
Critères d'acceptation	Bascule favori / non-favori • Connexion requise • Persisté en base, par une relation entre l'utilisateur et l'offre
Priorité	Must

Trois lignes qui contraignent déjà le développement. L'acteur dit qui a le droit, et la connexion requise en découle. L'action dit ce que fait l'interface : une bascule, donc un seul bouton qui change d'état, et non deux. Le bénéfice dit pourquoi la donnée doit survivre à la déconnexion : garder de côté suppose une table en base, pas un cookie de navigateur.

Les critères d'acceptation, eux, disent à quelle condition la user story est terminée. Recopiés dans l'issue correspondante, ils permettent de la fermer sur un fait plutôt que sur une impression.

La priorisation MoSCoW est un ajout, et c'est volontaire. Le format de la user story décrit un besoin ; il ne dit rien de son rang. J'ai donc ajouté au backlog une colonne de priorité empruntée à la méthode MoSCoW : must, should, could, won't. L'ajout est courant, mais il répond ici à un besoin précis : rendre les arbitrages lisibles d'un seul regard, sur un projet mené seul où personne ne vient les rappeler.

Voir l'annexe 1

4.2 Cas d'utilisation

Trois acteurs humains, et quatre acteurs secondaires. Le diagramme de cas d'utilisation, en notation UML, met en scène ce que chacun peut faire. Les quatre acteurs secondaires ne sont actionnés par personne : la

tâche planifiée qui lance l'agrégation à heure fixe, les interfaces des sources d'offres, le service d'envoi d'emails et le stockage des documents.

Acteur	Ce qu'il peut faire
Visiteur	Consulter les offres, chercher, filtrer, s'inscrire, se connecter.
Candidat	Tout ce que fait le visiteur, plus les favoris, les candidatures, le suivi, le profil et ses documents.
Administrateur	Tout ce que fait le candidat, plus la gestion des sources, des offres et des comptes, le tableau de bord et le déclenchement manuel d'une synchronisation.

Les trois acteurs s'emboîtent. Le candidat est un visiteur connecté, l'administrateur est un candidat doté de droits supplémentaires. Cet emboîtement n'est pas un raffinement de diagramme : c'est exactement ce que fait le contrôle d'accès du code, où deux middlewares se superposent : l'un vérifie qu'une session existe, l'autre que le rôle vaut **admin**. La conception et l'implémentation disent ici la même chose.

[Voir l'annexe 2](#)

>_ 5. Conception de la base de données

Le modèle de données a été dessiné avant la première table. Sept entités, leurs associations et leurs cardinalités ont été posées à l'étape 0, avant qu'une seule ligne de SQL soit écrite. Cette section explique la méthode suivie, ce que chaque passage d'un niveau au suivant a tranché, et ce que cette discipline a rapporté.

5.1 Pourquoi Merise

Merise sépare trois niveaux, et c'est là tout son intérêt. Le conceptuel décrit le métier sans rien savoir de la technique. Le logique le traduit en relations, sans encore choisir de moteur. Le physique l'exprime dans le dialecte d'un SGBD précis. Chaque niveau répond à des questions que le précédent n'avait pas à poser.

Niveau	Ce qu'il ajoute
Conceptuel	Les entités, leurs propriétés, leurs associations et leurs cardinalités.
Logique	Les clés primaires et étrangères ; les associations non fonctionnelles deviennent des relations.
Physique	Les types MySQL, les contraintes, les index, les comportements en cascade, le moteur.

Le raccourci existait, et il coûte plus tard. Créer les tables directement dans le SGBD aurait fait gagner une journée. Mais les clés étrangères s'ajoutent alors au fil des besoins, et les questions de suppression se règlent dans le code, au cas par cas. Merise oblige à y répondre avant, une seule fois, et au bon endroit.

Merise plutôt qu'un diagramme de classes. Le second décrit un modèle objet, avec son héritage et ses comportements. Ce projet n'a aucune classe : les modèles y sont des modules qui exposent des fonctions, et les données sont relationnelles de bout en bout. Décrire en objet ce qui sera stocké en relationnel aurait ajouté une traduction sans rien clarifier.

5.2 Du MCD au MLD, puis au MPD

Le verbe se lit dans les deux sens. Chaque association porte un verbe (**METTRE EN FAVORI, PROVENIR DE**) qui relie deux entités et s'énonce dans un sens comme dans l'autre : un utilisateur met en favori des offres, une offre est mise en favori par des utilisateurs. C'est cette lecture qui donne les cardinalités.

Les cardinalités décident du modèle logique. Un côté à 1,1 fait migrer l'identifiant de l'entité opposée en clé étrangère. Deux côtés à n l'interdisent : il faut une table de liaison, dont la clé primaire est la concaténation des deux identifiants. Sept associations, dix cardinalités 0,n pour quatre 1,1, donc quatre clés étrangères et trois tables de liaison, dont deux portent leurs propres colonnes : la date d'ajout d'un favori, l'état d'une candidature.

La clé étrangère, elle, sert les requêtes. C'est sur elle que la jointure s'appuie : une offre et sa source, un candidat et ses favoris se lisent en une seule requête au lieu de plusieurs allers-retours. Le modèle conceptuel décide donc déjà de la forme des requêtes qu'on écrira.

La base arbitre les accès concurrents. Deux clics simultanés sur le même bouton favori ne peuvent pas créer deux lignes : la clé primaire (**id_utilisateur, id_offre**) l'interdit. Ce n'est pas l'application qui se protège, c'est la base qui refuse.

Le passage au physique tranche le reste. Types MySQL, moteur InnoDB, encodage utf8mb4, et surtout les comportements qu'un modèle conceptuel ne pose pas : supprimer un compte efface ses documents et ses favoris en cascade, supprimer une offre encore rattachée à une candidature est refusé. Ces réponses sont écrites dans le schéma, pas dans le code.

Le modèle physique est devenu un script. `sql/schema.sql`, versionné dans le dépôt et rejoué à l'identique en local, en test, dans le conteneur et sur le serveur.

Les modèles : annexes 3 à 5

Le script : annexes 6 et 7

5.3 Ce que la méthode a apporté

L'intégrité est confiée à la base, pas au code. C'est la conséquence la plus durable des trois niveaux : chaque fois qu'une règle peut s'exprimer en contrainte, elle l'est. L'application ne peut alors pas la contourner, même par erreur, et la règle survit à toute réécriture du code.

La méthode a forcé des décisions avant le code. Le dédoublement en est l'exemple : la contrainte d'unicité ne porte pas sur la référence d'une annonce, mais sur le couple (**source, référence**). Deux fournisseurs peuvent numéroter leurs annonces de la même façon. Cette question s'est posée en dessinant le modèle logique, pas en découvrant des doublons en production.

Elle a aussi un coût, et il s'est présenté. Le 12 juillet, entre le back et le front de l'étape 5, France Travail a renvoyé un code de contrat **MIS** que le modèle n'avait pas prévu. Ajouter la valeur **interim** a demandé de reprendre le modèle physique, puis le script de création, puis la normalisation, dans cet ordre. Trois lignes modifiées dans deux fichiers versionnés, tracées par la branche **feat/contrat-interim** et la pull request 129.

>_ 6. Conception des interfaces

Les écrans ont été dessinés après le modèle de données, et avant le premier template. C'est l'ordre des issues G, H, I et J de l'étape 0 : la charte graphique, puis les wireframes, puis les maquettes, puis le diagramme de navigation.

Cet ordre a une logique. On fixe d'abord le vocabulaire visuel, ensuite on place les blocs sans se soucier de leur apparence, ensuite seulement on habille. Commencer par les maquettes revient à discuter des couleurs d'un bouton avant de savoir s'il a sa place sur l'écran.

6.1 Charte graphique

Le thème sombre est le thème principal. Le choix vient du public visé : des développeurs, qui passent leurs journées dans des éditeurs sombres.

L'interface reprend les codes de l'éditeur de code. Ceux de Visual Studio Code, dans lequel le projet a lui-même été écrit : un fond bleu très sombre plutôt qu'un gris neutre, un logo composé comme une invite de commande (>_ **MonPremierPoste**_, curseur clignotant compris) et du texte qui n'a pas une seule couleur : la menthe pour ce sur quoi on peut cliquer, le gris pour les informations secondaires, le blanc cassé pour le texte à lire.

Rôle	Thème sombre	Thème clair
Fond de page	#061018	#F4F8F7
Surface (carte, champ)	#0C1A22	#FFFFFF
Accent	#2DD4BF (menthe)	#0E9384 (teal)
Texte	#E6EDF3	#0B2027
Texte secondaire	#8FA3AE	#5A7184

Les polices sont auto-hébergées. Six fichiers **woff2** dans **public/fonts**, 137 Ko en tout, servis par l'application. C'est une dépendance extérieure de moins et une connexion de moins à établir avant le premier affichage : la page s'affiche dans sa typographie même si un service tiers est indisponible.

6.2 Wireframes

Le wireframe met les user stories à l'épreuve d'un écran. Le backlog énumère des besoins ; le wireframe les met en situation. Chaque story doit trouver sa place quelque part, et celles qui n'en trouvent pas signalent un oubli : un écran qui manque, ou un besoin formulé sans support. C'est autant un contrôle qu'un dessin.

Une première version du site, en gris. Des rectangles, du texte d'attente, aucune couleur ni typographie. L'habillage viendra plus tard : à ce stade, un écran ne se juge que sur ce qu'il contient et sur la hiérarchie de ses informations.

draw.io, pour trois raisons. Il est simple, ses moyens correspondent à ce qu'on attend d'une basse fidélité, et il s'ouvre directement dans l'éditeur : les wireframes se consultent sans quitter le projet.

Voir les annexes 8 et 9

6.3 Maquettes

La maquette sert de contrat. Une maquette validée devient la référence contre laquelle la vue est comparée : dire qu'un écran est terminé cesse d'être une appréciation : ou bien il correspond, ou bien il en diffère, et l'écart doit être justifié.

Un écran est né en cours de route, et d'un problème technique. Les annonces ne peuvent pas être indexées : elles proviennent des plateformes sources, et elles disparaissent dès qu'elles y sont retirées. La page de conseils sur la recherche d'un premier poste a été écrite pour deux raisons : accompagner les candidats dans leur démarche, et surtout donner au site un contenu qui lui appartienne et que les moteurs puissent indexer.

La haute fidélité, cette fois. Là où le wireframe ne montrait que la structure, la maquette l'habille : elle reprend les écrans tracés en gris et leur applique la charte : couleurs, typographie, rayons d'angle, états de survol et de focus. Chaque écran du parcours y figure, dans les deux thèmes et dans les trois formats. Plus aucune décision structurelle n'est ouverte à ce stade.

Figma, pour trois raisons. Il gère les composants et leurs variantes, donc les deux thèmes d'un même écran restent liés au lieu d'être deux dessins séparés. Les polices réelles du projet (**Inter** et **JetBrains Mono**) s'y installent, donc la maquette montre ce que le navigateur affichera. Et les valeurs s'y relèvent au lieu de s'estimer à l'œil : une couleur, un espacement, une taille se lisent dans l'outil et se recopient dans la feuille de style.

Ce qui se répète n'est dessiné qu'une fois. La plupart des éléments d'un écran se retrouvent sur les suivants : l'en-tête, le pied de page, les boutons, les puces, la carte d'offre. Chacun est défini une fois, avec les couleurs et la typographie de la charte, puis repris à l'identique partout où il apparaît. C'est autant de travail qui n'est pas refait, et autant d'occasions d'écart en moins entre deux écrans.

[Voir les annexes 10 à 12](#)

6.4 Diagramme de navigation

Les maquettes montrent des écrans ; elles ne disent pas comment on passe de l'un à l'autre. Des écrans dessinés un par un ne forment pas encore un site : il manque les chemins. Le diagramme de navigation est le seul document de conception qui les montre tous en même temps.

[Voir l'annexe 13](#)

>_ PARTIE 3 • RÉALISATION

>_ 7. Architecture et environnement de travail

La conception est finie, le code n'a pas commencé. Restent deux décisions à prendre avant la première ligne : comment le projet sera rangé, et avec quels outils il sera écrit.

7.1 Schéma d'architecture

Un serveur, deux conteneurs, trois services extérieurs. L'application tient sur une seule machine : Caddy reçoit les requêtes en HTTPS et les passe au conteneur Node.js, qui interroge un conteneur MySQL. Tout le reste appartient à quelqu'un d'autre : les trois interfaces d'offres d'emploi, l'envoi des courriels, l'hébergement des documents.

Voir l'annexe 14

7.2 Le MVC et le rôle des couches

Le MVC sépare ce qui s'affiche, ce qui décide et ce qui stocke. Express n'imposant aucune structure, il fallait en choisir une, et celle-là répartit les rôles au lieu de les mélanger. Un développeur qui reprendrait le projet sait alors où chercher sans qu'on le lui explique : l'affichage dans les vues, les décisions dans les contrôleurs, les requêtes vers la base dans les modèles.

Couche	Rôle
routes	Associer une URL et un verbe HTTP (GET, POST, DELETE) à un contrôleur, précédé des middlewares et des validateurs déclarés pour cette route.
middlewares	S'exécuter dans la chaîne, avant le contrôleur, avec le pouvoir d'interrompre la requête. Ainsi requireAuth renvoie vers la page de connexion quand aucune session n'est ouverte : le contrôleur n'est jamais appelé.
controllers	Lire la requête, appeler ce qu'il faut, puis choisir la réponse (une vue, une redirection, du JSON ou un fichier) et son code HTTP.
services	La logique métier qui ne se résume pas à un accès aux données, écrite sans rien connaître de la requête HTTP.
models	Le seul endroit où l'application lit et écrit des données.
views	Produire le HTML à partir des données que le contrôleur et les middlewares leur passent.

Voir l'annexe 15

7.3 Environnement de développement

Un poste, quatre outils. Visual Studio Code comme éditeur, Node.js 22 comme moteur d'exécution, MySQL 8 comme serveur de données local, Git et GitHub pour la gestion de versions. Les échanges avec le dépôt distant sont authentifiés et chiffrés ; ceux avec le serveur de production passent par SSH, le détail des clés est en 12.4.

La version de Node est déclarée, pas subie. Le `package.json` exige Node 22 ou plus, et l'intégration continue installe cette version-là. Deux conséquences concrètes : le rechargement automatique en développement se fait avec `node --watch`, sans dépendance supplémentaire, et le client HTTP utilise le `fetch` natif plutôt qu'une bibliothèque. Deux dépendances évitées parce que la plateforme les fournit désormais.

La configuration vit hors du code. `dotenv` charge un fichier `.env` au démarrage, une seule ligne dans `server.js`. Ce fichier n'est pas versionné ; `.env.example` l'est, avec les mêmes clés et aucune valeur. Le premier tient les secrets hors du dépôt, le second dit à qui reprend le projet quelles variables renseigner. Sans l'exemple, la seule façon de découvrir une variable manquante serait de voir l'application échouer.

7.4 Qualité de code

Deux outils, deux métiers. ESLint cherche des erreurs : variable inutilisée, import mort, faute de frappe sur un identifiant ; Prettier impose une mise en forme unique. Le paquet `eslint-config-prettier` les réconcilie en désactivant les règles d'ESLint qui portent sur l'apparence : sans lui, chaque formatage rouvrirait des avertissements que le formatage venait de créer. Deux commandes, `npm run lint` et `npm run format`.

7.5 Choix techniques justifiés

Chaque dépendance devrait pouvoir se justifier en trois temps. Son rôle, la raison du choix, et ce qui a été écarté. Les dépendances qui servent un domaine précis sont justifiées là où elles servent : `bcrypt` en 10.1, `csrf-sync` en 10.2, Jest en 11.2. Cette section porte les choix structurants, ceux qui ont été faits avant la première ligne et qui engagent tout le reste.

Choix	Pourquoi	Écarté
Node.js et Express 5	Un seul langage du navigateur au serveur ; Express est minimal et n'impose pas de structure.	NestJS, un framework complet qui aurait décidé de l'architecture à ma place.
EJS, rendu côté serveur	Branché sur le moteur de vues d'Express en une ligne, sans étape de compilation. Le serveur envoie du HTML complet : référencement naturel, page utilisable sans JavaScript.	React, qui aurait imposé une API séparée, ou Next.js et sa chaîne de compilation pour que les pages restent indexables.
MySQL, relationnel	Les données sont fortement liées, et l'intégrité doit être garantie par la base.	MongoDB, qui ne gère pas les clés étrangères : rien n'empêcherait de supprimer une offre à laquelle une candidature est encore rattachée.
mysql2, sans ORM	Le SQL est écrit à la main, donc lu, compris et optimisable ; le driver gère les requêtes préparées.	Sequelize, dont la valeur apparaît sur des schémas mouvants et de grandes équipes.
Un monolithe	Le découpage en couches sépare déjà les responsabilités.	Des micro-services, qui n'auraient séparé que les déploiements.

Pas d'ORM, et pourquoi. Un ORM fait correspondre des tables et des objets, et écrit le SQL à la place du développeur. C'est exactement ce qui le disqualifiait ici : la requête réellement exécutée devient celle de l'outil, et ce que je saurais en dire s'arrêterait à l'appel de méthode. Sur un schéma de dix tables, stabilisé avant le développement et tenu par une seule personne, il n'aurait rien apporté d'autre qu'une couche entre moi et mes requêtes. Les écrire à la main m'oblige à comprendre ce que la base exécute ; dans un projet de formation, c'est le but, pas une contrainte.

>_ 8. Implémentation du back

Dix modèles, onze services, trois connecteurs de sources. Le serveur lit et écrit en base, applique les règles métier, et va chercher les offres là où elles sont publiées.

8.1 Accès aux données

Le SQL est écrit à un seul endroit. Dix modèles, un par table principale, et la règle est absolue : aucun contrôleur, aucun service, aucune vue ne contient de requête. Chercher où une table est lue revient à ouvrir un fichier.

Le pool de connexions est partagé. Un seul pool est créé au démarrage et importé par tous les modèles. Ouvrir une connexion par requête aurait coûté une poignée de main réseau à chaque affichage de page ; le pool les recycle, plafonnées à dix simultanées.

```
1 // MySQL driver, promise flavour (async/await).
2 import mysql from "mysql2/promise";
3
4 // MySQL connection pool shared by the models.
5 const pool = mysql.createPool({
6   host: process.env.DB_HOST,
7   port: Number(process.env.DB_PORT),
8   user: process.env.DB_USER,
9   password: process.env.DB_PASSWORD,
10  database: process.env.DB_NAME,
11  waitForConnections: true, // wait when the pool is full
12  connectionLimit: 10, // max simultaneous connections
13  queueLimit: 0, // unlimited queue
14  charset: "utf8mb4",
15 });
16
17 // Checks the database is reachable (called at startup).
18 export async function testConnection() {
19   const connection = await pool.getConnection();
20   try {
21     await connection.ping();
22     console.log("✅ Connexion MySQL établie");
23   } finally {
24     connection.release();
25   }
26 }
27
28 // Pool exported for the models (prepared statements).
29 export default pool;
```

Figure 3. src/config/db.js : le pool partagé, sa configuration lue dans l'environnement, et le test de connexion joué au démarrage.

Ce que dit la configuration. Aucune valeur n'est écrite dans le fichier : l'adresse du serveur, le port, le compte et le nom de la base sont lus dans l'environnement. Trois réglages méritent un mot. **waitForConnections** fait qu'une requête attend son tour quand les dix connexions sont toutes prises, au lieu d'échouer ; **queueLimit** à zéro ne plafonne pas cette file d'attente ; et **charset** déclare pour la connexion le même jeu de caractères que les tables, **utf8mb4**, sans quoi un intitulé d'offre accentué reviendrait abîmé, et les offres viennent d'interfaces sur lesquelles je n'ai aucune prise.

La connexion est éprouvée avant que le serveur écoute. `testConnection` emprunte une connexion au pool, la teste par un ping, et la rend dans un `finally` : même si le test échoue, elle retourne au pool. Le démarrage s'interrompt si la base ne répond pas : mieux vaut ne pas démarrer que servir des erreurs.

Deux écritures concurrentes ne sont pas arbitrées, elles sont rendues sans effet. Ajouter un favori deux fois (deux clics rapprochés, ou deux onglets ouverts sur la même offre) ne produit ni doublon ni erreur. L'insertion est écrite en **INSERT IGNORE** : si la ligne existe déjà, la base ne fait rien et l'appel réussit quand même. Aucun verrou applicatif n'intervient : c'est la clé primaire composite décrite en 5.2 qui tranche, et l'arbitrage revient à la base plutôt qu'au code.

8.2 Trois sources, un seul format

Trois sources, trois interfaces, un seul format interne. France Travail, Adzuna et La Bonne Alternance ne se ressemblent en rien : authentification différente, pagination différente, noms de champs différents, unités de salaire différentes. Le catalogue, lui, n'a qu'une forme.

La conversion est confiée à un connecteur par source. Les trois exportent le même contrat (une seule fonction) et gardent le reste pour eux : l'obtention d'un token d'accès pour l'un, la construction des adresses de pagination pour un autre, la conversion des champs pour chacun. Deux registres font la liaison, l'un vers le connecteur, l'autre vers le normaliseur.

```
src/services/sources/

    adzuna.js                                franceTravail.js                        labonnealternance.js
112 export default {                        122 export default {                    65 export default {
113   rechercherOffres,                    123   rechercherOffres,                66   rechercherOffres,
114 };                                      124 };                                67 };

src/services/synchronisationService.js
13 // API connector to use for each source, looked up by its name in the database.
14 const CONNECTEURS = {
15   "France Travail": franceTravail,
16   Adzuna: adzuna,
17   "La Bonne Alternance": laBonneAlternance,
18 };

47 const connecteur = CONNECTEURS[source.nom_source];
```

Figure 4. Le même contrat exporté par les trois connecteurs, le registre qui les associe à leur source, et la ligne qui en choisit un.

Les trois connecteurs sont donc interchangeables. La synchronisation lit le nom de la source en base, prend le connecteur correspondant et l'appelle sans savoir lequel elle tient : c'est ce que la programmation orientée objet appelle le polymorphisme, et ce que chacun garde pour lui, l'encapsulation. L'un et l'autre passent ici par le module plutôt que par la classe. L'héritage, lui, n'avait rien à transmettre : les trois connecteurs partagent un contrat, pas du code.

La collecte se déroule toujours dans le même ordre.

- Appel paginé de l'interface, avec un délai entre deux pages calé sur le quota du fournisseur.
- Normalisation : chaque annonce prend la forme interne, le département devient une région, le type de contrat est ramené à la liste des valeurs connues, et la période du salaire, qu'une des sources ne fournit pas, est déduite de l'ordre de grandeur du montant. Les salaires seulement estimés par une source sont écartés.

- Filtrage de pertinence, décrit en 8.3 : les annonces qui n'ont rien à faire dans un site pour développeurs sont écartées.
- Détection junior, calculée à l'entrée : le composant est décrit en 8.3.
- Enregistrement, où la contrainte d'unicité sur le couple source et référence externe fait le dédoublement.
- Rattachement des technologies détectées dans le texte.

Le dédoublement est délégué à la base. Il n'est pas fait en mémoire, ni par une lecture préalable : l'écriture est tentée, et la contrainte tranche. Une annonce déjà connue est mise à jour au lieu d'être ajoutée, ce qui rafraîchit au passage sa date de dernière vue : c'est cette date qui permettra plus tard de repérer son retrait.

Un troisième registre porte les obligations légales. Les licences des trois sources se contredisent : l'une interdit la reproduction de son logo, une autre l'impose, la troisième demande la mention de sa paternité. Plutôt que d'écrire ces différences dans les templates, chaque source déclare les siennes dans sa configuration : badge d'attribution, mention de licence, délai d'anonymisation, rythme de collecte et quotas.

Les vues lisent cette déclaration au lieu de connaître les sources. La conséquence est la même que pour les connecteurs : ajouter un fournisseur consiste à décrire ses contraintes, pas à modifier une vue. Une contrainte juridique a ainsi produit un effet d'architecture : elle m'a obligé à traiter les sources comme des données de configuration plutôt que comme du code dupliqué.

Chaque collecte laisse une trace. Nombre d'annonces récupérées, ignorées, en doublon, en erreur, avec la date et l'origine du déclenchement. Cette table alimente le tableau de bord d'administration et rend une anomalie lisible sans ouvrir les journaux du serveur.

Les trois sources ne se ressemblent sur aucun point. Ce tableau réunit ce que chacune impose, et explique pourquoi la configuration a dû être déclarée source par source.

	Adzuna	France Travail	La Bonne Alternance
Authentification	Deux identifiants applicatifs, passés dans l'adresse	Token OAuth 2.0, renouvelé avant expiration	Clé d'API, dans l'en-tête
Pagination	80 pages de 50 annonces	21 pages de 150 annonces	Aucune : un seul appel, national
Plafond par passe	4 000 annonces	3 150 annonces, profondeur maximale de l'API	150 annonces
Quotas	25 par minute, 250 par jour, 2 500 par mois	4 appels par seconde	60 appels par minute
Licence	Attribution obligatoire, logo imposé	Logo interdit, mentions textuelles, anonymisation après retrait	Licence Ouverte 2.0, attribution simple

8.3 Le filtrage des annonces

Le composant métier central du projet décide si une offre est ouverte à un débutant. C'est la promesse du site, et la seule règle que personne ne pouvait me donner toute faite. Collecter, afficher, candidater : tout le reste existe pour amener cette décision jusqu'au candidat.

La décision suit une priorité stricte, et l'ordre compte autant que les critères. Une alternance ou un stage qualifient d'emblée. Vient ensuite un veto qui ne regarde que le titre : un terme d'encadrement dans l'intitulé désigne un poste senior, quoi que raconte la description. Puis un second veto sur le texte entier, déclenché

par un terme de séniorité ou par une durée d'expérience supérieure à trois ans : les durées chiffrées sont lues, pas seulement les mots. Ce n'est qu'après ces deux vetos que les signaux favorables comptent.

```
55 // Decides whether a posting is open to juniors, with a strict precedence.
56 // experienceExige: France Travail code ("D" beginner, "S" wished, "E" required); null elsewhere.
57 export function estJunior({ titre, description, typeContrat, experienceExige } = {}) {
58   // 1. Work-study or internship: qualify outright.
59   if (typeContrat === "alternance" || typeContrat === "stage") return true;
60
61   // 1 bis. Senior title veto: a management term IN THE TITLE defines a senior role, whatever the
62   // description says. Searched on the title alone, not on the whole text.
63   if (REGEX_NEGATIFS_TITRE.test(normaliser(titre ?? ""))) return false;
64
65   const texte = normaliser(`${titre ?? ""} ${description ?? ""}`);
66   const annees = signauxAnnees(texte);
67
68   // 2. Senior veto: a seniority keyword OR a duration >= the threshold.
69   if (REGEX_NEGATIFS.test(texte) || annees.senior) return false;
70
71   // 3. Junior signal: "débutant accepté" (FT) OR a positive keyword OR a duration below it.
72   if (experienceExige === "D" || REGEX_POSITIFS.test(texte) || annees.junior) return true;
73
74   // 4. No signal: junior is not presumed (false by default, the column must discriminate).
75   return false;
76 }
```

Figure 5. La détection des offres ouvertes aux juniors, dans `src/services/juniorService.js` : les quatre étapes de la priorité sont numérotés dans le code.

Sans aucun signal, la réponse est non. Une annonce qui ne dit rien de l'expérience attendue n'est pas présumée ouverte aux débutants : une qualification qui ne tranche pas ne vaut rien. Le dictionnaire est par ailleurs bilingue (trente-deux termes favorables, vingt-trois défavorables) parce que l'une des sources sert des annonces rédigées en anglais. La fonction n'accède à rien : un titre, une description, une décision, que dix-huit tests unitaires vérifient étage par étage. Son verdict est calculé une fois, à la collecte, et rangé dans une colonne que la recherche lit ensuite.

La plus grosse difficulté du projet n'a pas été de décider, mais de trier. La vue junior s'est longtemps remplie d'annonces qui n'étaient pas des offres de développement. La cause ne tenait pas à mon code : elle tient à la façon dont chaque source se laisse interroger.

Deux de mes trois sources se demandent par code métier. France Travail et La Bonne Alternance acceptent une liste de codes ROME, la nomenclature officielle des métiers ; j'en interroge trois, ceux du développement. Le tri est donc fait chez elles, avant l'envoi : ce qui me revient relève déjà du développement.

Adzuna, elle, ne connaît que la recherche par mots-clés. Elle me renvoie toute annonce contenant le mot demandé, sans savoir de quel métier il s'agit. Or « développement » ne désigne pas toujours du logiciel : j'ai reçu de sa part des offres de développement des ressources humaines, de développement commercial, et jusqu'à une annonce de développement pour une maison de souliers. Chez elle, le tri ne pouvait se faire qu'après réception, c'est-à-dire chez moi.

Les mots-clés vivent dans deux fichiers de configuration, séparés du code. `src/config/junior.js` porte les termes de la détection junior ; `src/config/pertinence.js` ceux du tri, près de deux cents entrées réparties en six listes. Le filtre les lit en deux temps : un premier étage nomme les métiers à écarter, sur toutes les sources ; un second, réservé à Adzuna, exige en plus un terme technique dans le titre ou la description. Ajouter une règle revient à compléter une liste, jamais à toucher au code.

```

src/config/junior.js
4 export const MOTS_POSITIFS = [
5   // FR
6   "junior",
7   "débutant",
8   "débutant accepté",
9   :
26  // EN
27  "junior",
28  "entry level",
29  "graduate",
30  "new grad",
31  :
39 ];
40 :
83 export const SEUIL_ANNEES = 3;

src/config/pertinence.js
3 // Sources whose postings must carry a technical term to be kept.
4 export const SOURCES_A_FILTERER = ["Adzuna"];
5 :
85 // Roles discarded with no catch-up possible.
86 export const ROLES_HORS_CIBLE = [
87   "business develop",
88   :
89   "developpement commercial",
90   "developpeur commercial",
91   :
97   "product owner",
98   "product manager",
99   "scrum master",
100  :

```

Figure 6. Les deux fichiers de configuration : les mots-clés de la détection junior à gauche, ceux du tri à droite.

Le filtre n'intervient qu'à un seul endroit de la chaîne. À la collecte, avant l'enregistrement : une annonce écartée est comptée, puis abandonnée. Elle n'entre jamais en base, et aucune autre partie de l'application n'a donc à se demander si elle a sa place dans le catalogue.

```

src/services/pertinenceService.js
36 export function estPertinente({ titre, description } = {}, nomSource) {
37   :
44   // Out-of-scope roles, before any catch-up.
45   if (contient(t, ROLES_HORS_CIBLE)) {
46     return false;
47   }
48   :
54   // Sources that classify by occupation code: kept as they are.
55   if (!SOURCES.has(nomSource)) {
56     return true;
57   }
58   :
63   // Otherwise a technical term must appear in the title or the description.
64   return REGEX_TECHNIQUE.test(normaliser(`${titre ?? ""} ${description ?? ""}`));
65 }

src/services/synchronisationService.js
71 // Out-of-scope roles: neither inserted nor refreshed.
72 if (!estPertinente(offre, source.nom_source)) {
73   nbIgnorees++;
74   continue;
75 }
76
77 const { id, insere } = await Offre.upsert(source.id_source, offre);

```

Figure 7. Le filtre de pertinence et son unique point d'application : l'annonce écartée est comptée, puis abandonnée avant l'enregistrement.

8.4 Tâches planifiées et garde-fous

Les collectes partent seules, à des heures choisies. Une planification par source, réglée dans sa configuration, plus deux tâches d'entretien.

Tâche	Fréquence	Raison de l'heure
-------	-----------	-------------------

Collecte Adzuna	1 fois par jour, 3 h	Le quota contractuel du plan gratuit.
Collecte France Travail	3 fois par jour, 6 h, 12 h et 18 h	La licence impose un appel au moins toutes les 24 h et l'application des retraits.
Collecte La Bonne Alternance	3 fois par jour, 7 h, 13 h et 19 h	Aucune contrainte de quota : les créneaux sont simplement décalés d'une heure sur ceux de France Travail.
Entretien du catalogue	1 fois par jour, 5 h	Après la collecte Adzuna, avant les deux autres.
Purge des jetons expirés	1 fois par jour, 3 h 30	Décalée des collectes, pour que les journaux ne se mélangent pas.

Une offre passe par trois états, et un seul est définitif. Chaque collecte rafraîchit la date de dernière vue des annonces qu'elle revoit ; celles que la source ne renvoie plus gardent une date ancienne et passent à l'état expiré. L'annonce quitte alors les résultats de recherche, mais elle reste dans les favoris et dans les candidatures, marquée « Expirée », et sa page de détail propose encore de postuler malgré l'expiration. L'état est réversible : une collecte qui la revoit la réactive.

L'entretien quotidien tranche, et l'ordre de ses trois opérations est une décision. Les favoris qui pointent vers une offre expirée sont retirés d'abord : c'est ce qui libère l'offre de toute référence. Les offres que plus rien ne retient sont alors supprimées. Restent celles qu'une candidature rattache : elles ne peuvent pas disparaître sans faire perdre au candidat la trace de sa démarche, elles sont donc anonymisées.

```

src/services/maintenanceOffres.js
74 export async function entretenirCatalogue() {
75   const nbFavorisRetires = await retirerFavorisOffresExpirees();
76   const nbPurgees = await purgerOffresExpirees();
77   const nbAnonymisees = await anonymiserOffresRetirees();
78   :
79
src/models/offre.js
204 // Deletes the expired postings that no favorite and no application references. The others are
205 // kept (FK ON DELETE RESTRICT): a posting one has applied to must stay readable.
206 async function purgeUnreferenced() {
207   const [result] = await pool.execute(
208     `DELETE FROM offre
209      WHERE statut_offre = 'expiree'
210      AND NOT EXISTS (SELECT 1 FROM favori WHERE favori.id_offre = offre.id_offre)
211      AND NOT EXISTS (SELECT 1 FROM candidature WHERE candidature.id_offre = offre.id_offre)`
212   );
213   :

```

Figure 8. L'entretien du catalogue : les trois opérations dans l'ordre où elles s'exécutent, puis la requête de purge et les trois conditions qu'une offre doit remplir pour être supprimée.

Supprimer dès la première absence aurait été plus simple mais dangereux. Une collecte à moitié ratée ne renvoie qu'une fraction du catalogue : tout le reste paraîtrait retiré, et une suppression immédiate effacerait le catalogue d'une source entière, sans retour. L'état expiré donne le droit à l'erreur ; un garde-fou la rend improbable : l'expiration est suspendue tant que la collecte n'a pas enregistré au moins la moitié du catalogue

de la source. Il ne se contente pas de ne rien faire, il le fait savoir : une anomalie est remontée, et le catalogue reste en l'état.

Un délai encadre l'anonymisation. Une annonce retirée n'est anonymisée qu'après deux jours d'expiration, soit six collectes consécutives pour la source la plus fréquente. Une interface momentanément défaillante ne peut donc pas déclencher une opération qu'aucune collecte ultérieure ne pourrait défaire.

>_ 9. Implémentation du front

Quand une donnée arrive dans une balise `<%= %>`, tout a déjà été décidé. La route a reçu, le contrôleur a tranché, le modèle a lu ; le template ne fait plus qu'écrire. Aucune règle métier ne descend jusque-là.

9.1 Interfaces statiques

Une seule feuille de style, sept breakpoints, aucun framework. Les règles de base décrivent l'affichage sur grand écran ; ils replient ensuite la mise en page : 1180, 1120, 1040, 860, 820, 760 et 640 pixels. Ce ne sont pas des largeurs d'appareils : un breakpoint mesure la fenêtre, et une fenêtre fait rarement la taille de l'écran : un navigateur ouvert à côté d'un document, une tablette en écran partagé. Ni bibliothèque d'icônes, ni image décorative : l'éco-conception se joue surtout dans ce qu'on ne charge pas.

Les vues sont découpées en partiels. Un partial est un fragment de template que plusieurs pages incluent au lieu de le recopier : le head et ses métadonnées, le header et sa navigation, le footer et ses mentions légales, les messages flash qui annoncent le résultat d'une action. La carte d'une offre en est un aussi : le même fichier sert la liste des offres, la page des favoris et la page d'accueil, et la corriger une fois la corrige aux trois endroits.

9.2 Partie dynamique de l'interface

La liste des offres change sans recharger la page. Taper une recherche ou cocher un filtre met la liste à jour sur place ; le reste de l'écran ne bouge pas. Le script demande au serveur la liste correspondante, puis remplace l'ancienne par celle qu'il reçoit.

Le navigateur ne parle jamais à la base. Il envoie une intention, et le serveur décide : la route, puis le contrôleur qui interroge le modèle, puis une requête SQL préparée. Ce qui revient n'est pas une donnée, c'est du HTML : le contrôleur reconnaît la demande du script et rend alors les seuls résultats au lieu de la page entière, mais avec le même gabarit, celui que la page complète inclut déjà. La liste n'est donc écrite qu'une fois, et le navigateur ne fabrique aucune mise en page. Si la demande échoue, il recharge la page comme un lien ordinaire l'aurait fait : le script améliore un comportement qui existe déjà, à deux exceptions près que je préfère nommer : mettre une offre en favori et ouvrir le menu de navigation sous 760 pixels demandent JavaScript ; consulter, chercher, filtrer et candidater s'en passent.

```

public/js/offres.js – le navigateur demande
40   cible.searchParams.set("partial", "1"); // Asks the controller for the fragment version.

src/routes/offres.js
7   // GET /offres: paginated public list.
8   router.get("/", liste);

src/controllers/offreController.js
48   const { resultats, total } = await Offre.search(filtres, { page, limite: PAR_PAGE });
:
83   // fetch request: only the list + pagination fragment is returned. The full page is the no-JS base.
84   if (req.query.partial === "1") {
85     return res.render("offres/_resultats", donnees);
86   }
87   res.render("offres/liste", donnees);

src/views/offres/_resultats.ejs
14   <% offres.forEach((offre) => { %>
15     <%- include("_carte", { offre, montrerJunior: voirTout }) %>

public/js/offres.js – et le remet en place
54   resultats.innerHTML = html;

```

Figure 9. Le trajet complet d'un filtre : le navigateur demande le fragment, la route mène au contrôleur, le modèle interroge la base, le gabarit écrit les cartes, et le script remet le tout en place.

9.3 Accessibilité mise en œuvre

Un outil fait pour rendre des candidats visibles ne peut pas en exclure. Chercher un emploi ici, c'est remplir un formulaire, parcourir une liste, comparer des annonces, trois actions qui ne reposent que sur du texte.

- Chaque page a une ossature explicite : un <h1> unique, une hiérarchie de titres qui suit le contenu, et les repères <header>, <nav>, <main> et <footer>, que les lecteurs d'écran savent atteindre directement sans parcourir ce qui les précède.
- Le site se parcourt au clavier : une règle :focus-visible dessine un anneau autour de l'élément atteint (au clavier seulement, jamais au clic), et un lien d'évitement mène droit au <main> au lieu de retraverser l'entête, huit à onze arrêts à chaque page.
- Chaque champ de formulaire porte une étiquette liée, l'aide de saisie est rattachée au champ qu'elle concerne, et un champ en erreur reçoit l'attribut aria-invalid en plus du message affiché.
- Aucune information ne repose sur la couleur seule : un contrat, un statut, une offre expirée ou ouverte aux juniors portent un mot écrit, et la couleur ne fait que le confirmer.
- Là où HTML n'a pas d'équivalent, ARIA prend le relais, et seulement là : aria-pressed dit l'état d'un bouton à bascule, aria-live fait annoncer une zone mise à jour sans rechargement, aria-hidden retire le décor de ce que lisent les lecteurs d'écran.

Le rouge d'un champ en erreur ne dit rien à un lecteur d'écran. Une personne qui voit l'écran repère le champ fautif d'un coup d'œil ; un lecteur d'écran, lui, avance champ par champ, et la couleur ne lui parvient pas. Sans indication, il annonce « Adresse email, champ de saisie », exactement ce qu'il annonçait avant l'erreur. Avec l'attribut, il annonce « Adresse email, champ de saisie, invalide », au moment où la personne arrive sur le champ. L'attribut n'est pas écrit à la main, vue par vue : une fonction posée une fois côté serveur le calcule, et les vues l'appellent sur chaque champ, vingt champs répartis dans huit formulaires. Elle le calcule à partir de la même liste d'erreurs que celle qui affiche les messages, et c'est tout l'intérêt : il ne peut pas y avoir un champ signalé à l'écran et annoncé valide au lecteur d'écran.

```

src/app.js - Le serveur fabrique l'attribut
58 // --- Global view helper: flagging a field in error (RGAA) ---
59 // The forms already list their errors; aria-invalid designates THE faulty field for a screen
60 // reader going field by field. Render with <%- %>: the result is an attribute, never user data.
61 app.locals.attributInvalide = (erreurs, champ) =>
62   Array.isArray(erreurs) && erreurs.some((erreur) => erreur.path === champ) ? 'aria-invalid="true"' : "";

— Le HTML reçu par le navigateur, champ en erreur —
<label for="email">Adresse email</label>
<input type="email" id="email" name="email" required aria-invalid="true" />

```

Figure 10. L'attribut est fabriqué une fois côté serveur, à partir de la même liste d'erreurs que les messages affichés ; les vues l'appellent sur chaque champ. Un lecteur d'écran annonce alors « Adresse email, champ de saisie, invalide ».

9.4 Référencement

Un agrégateur qu'on ne trouve pas ne sert personne. Le robot d'un moteur reçoit exactement ce que reçoit un visiteur : la page part complète du serveur. Reste à lui dire ce qu'il peut en faire, et ce qu'il doit laisser.

Élément	Rôle
Titre et description par page	Chaque page publique fournit les siens ; aucun titre générique.
Adresse canonique	Toutes les combinaisons de filtres de /offres convergent vers une seule adresse.
robots.txt	Écarte les zones privées (/admin, /profil, /favoris, /candidatures) et déclare l'adresse du sitemap.
sitemap.xml	Les sept adresses destinées à l'indexation ; les offres et les pages légales en sont absentes.
Page « plan du site »	Le même parcours en HTML, pour les visiteurs.
Page « Nos conseils »	La matière durable à indexer, les offres en étant écartées.
Métadonnées de partage	Titre, description, image et adresse pour un aperçu correct sur les réseaux.

L'adresse canonique est le point le plus important. Sur la liste des offres, les filtres voyagent dans l'adresse : chaque combinaison en produit une variante (/offres?region=paca&typeContrat=alternance) pour une page qui reste la même. Avec sept paramètres, elles se comptent par centaines, et un moteur y voit autant de pages distinctes. La balise canonique lui désigne l'adresse de référence, ici le chemin sans les filtres : /offres. Un middleware écrit pour l'occasion (quatre lignes à l'entrée de l'application) la calcule à chaque requête et la dépose dans les variables locales de la réponse : toutes les vues y ont accès sans qu'aucun contrôleur ait à la transmettre.

```

src/app.js
120 // req.path excludes the query string, so all the filter combinations of /offres converge on the
121 // same canonical address instead of becoming as many distinct pages for a search engine.
122 app.use((req, res, next) => {
123   res.locals.urlCanonique = `${(process.env.APP_URL ?? "").replace(/\$/ , "")}${req.path}`;
124   next();
125 });

src/views/partials/head.ejs
23 <!-- Keeps the page out of search engines when the view provides `noindex`. -->
24 <% if (typeof noindex !== "undefined" && noindex) { %>
25   <meta name="robots" content="noindex, follow" />
26 <% } %>
:
31 <link rel="canonical" href="<%= urlCanonique %>" />

src/views/offres/detail.ejs
1 <!-- noindex: the posting is duplicated from its source API, it is short-lived, and the France
2   Travail licence forbids keeping a withdrawn one in clear - an indexed page would survive in
3   the search cache. `follow` keeps the links crawlable. -->
:
7   noindex: true,

```

Figure 11. L'adresse canonique, de son calcul à son émission, et la consigne de non-indexation que la vue d'une offre déclare.

La liste est indexée ; les annonces, non. La canonique disait « référence ce contenu, mais à cette adresse » ; pour la page d'une annonce (/offres/1247), il s'agit de dire « ne le référence pas du tout ». Elle porte une balise noindex : elle demande au moteur de ne pas la référencer, tout en le laissant suivre les liens qu'elle contient. Trois raisons la justifient : l'annonce est reprise de l'interface de sa source, elle ne vit que quelques semaines, et la licence France Travail interdit de conserver en clair une offre retirée, or une page indexée lui survivrait dans le cache du moteur. Les deux pages légales portent la même balise.

Deux fichiers auraient pu contredire cette consigne. robots.txt n'interdit pas l'accès aux offres, et c'est délibéré : un robot à qui l'on refuse la page ne peut pas y lire le noindex qu'elle contient : la lui fermer reviendrait à l'empêcher d'apprendre qu'il ne doit pas l'indexer. sitemap.xml, lui, ne les liste pas : proposer à l'indexation une page marquée « à ne pas indexer » enverrait deux signaux contraires.

Écarter les offres pose une question : que reste-t-il à indexer ? C'est le paradoxe d'un agrégateur : son contenu principal est celui qu'il ne peut pas exposer. D'où une page écrite pour être trouvée : « Nos conseils » dans le footer, mais « Comment décrocher son premier poste de développeur » pour un moteur : le libellé court sert la navigation, le titre long répond à ce qu'un débutant cherche vraiment. Et il n'est pas seul : chacun des titres de section en est une autre entrée : ce qu'un recruteur regarde chez un débutant, alternance, stage, CDD ou CDI, le portfolio et le dépôt de code, le CV, la lettre et la relance, l'entretien technique. Autant de formulations qui durent, là où une annonce vit quelques semaines.

>_ PARTIE 4 • MISE À L'ÉPREUVE

>_ 10. Sécurité de l'application

Ce site ne détient ni argent ni numéro de carte. Il détient un CV, une lettre de motivation, et le fait qu'une personne cherche un emploi, une information qu'elle n'a pas forcément partagée avec son employeur actuel. Il écrit en outre aux entreprises au nom du candidat : prendre son compte, c'est prendre sa parole.

Les sous-sections suivent l'ordre où une requête traverse l'application : qui vous êtes, ce que vous envoyez, ce qui atteint la base, ce qui revient à l'écran, ce à quoi vous avez droit, puis ce qui protège l'ensemble quel que soit le chemin, et ce que la loi impose.

10.1 Mots de passe et sessions

Un compte suit toujours le même trajet. Il naît, il s'authentifie, il vit le temps d'une session, il s'éteint, et il arrive qu'on doive le récupérer. À aucun de ces moments le serveur ne conserve un secret lisible.

À l'inscription, douze caractères au minimum, mais la longueur ne suffit pas. Il faut aussi une majuscule, une minuscule, un chiffre et un caractère spécial. Les deux exigences sont indissociables. Le tableau ci-dessous donne le temps qu'il faudrait à un attaquant pour reconstituer un mot de passe à partir de sa forme hachée, stockée en base, en essayant les combinaisons une à une : un mot de passe de douze chiffres est retrouvé en trois mois, quand les mêmes douze caractères pris dans les quatre familles sont quasiment impossibles à retrouver.

Longueur	Chiffres seulement	Les quatre familles
8 caractères	Instantané	164 ans
10 caractères	1 jour	803 000 ans
12 caractères (la règle du site)	3 mois	3 milliards d'années

Ces durées supposent le pire : un attaquant est entré dans la base et en a emporté le contenu. Il n'y trouve pourtant aucun mot de passe lisible : ils sont tous hachés, et l'opération ne s'inverse pas : il ne lui reste donc qu'à deviner. Sur ses propres machines, sans limite de tentatives, il hache des candidats les uns après les autres jusqu'à en trouver un qui produise exactement le même résultat. C'est ce qu'on appelle une attaque hors ligne, et le tableau la mesure avec douze cartes graphiques de dernière génération, contre du bcrypt au coût 10, d'après Hive Systems (2025). Le site emploie le coût 12, soit quatre fois plus de calcul par essai.

Le mot de passe n'est ensuite plus jamais lisible. Il est haché avec bcrypt au coût 12, soit 2^{12} tours. Ce paramètre est un arbitrage : plus il est élevé, plus une attaque par force brute coûte cher, mais plus la connexion est lente pour l'utilisateur légitime.

Ce mot de passe haché porte un nom : l'empreinte. Celle que produit bcrypt fait soixante caractères et se lit en trois parties : la variante de l'algorithme, le coût, puis le sel suivi du condensat. Le sel n'a donc pas de colonne à lui : il voyage dans l'empreinte, et il est tiré au hasard à chaque enregistrement. Deux utilisateurs ayant choisi le même mot de passe produisent ainsi deux empreintes différentes, et vérifier une connexion ne consiste jamais à comparer deux chaînes : bcrypt relit le sel et le coût dans l'empreinte stockée, refait le calcul avec eux, et compare le résultat.

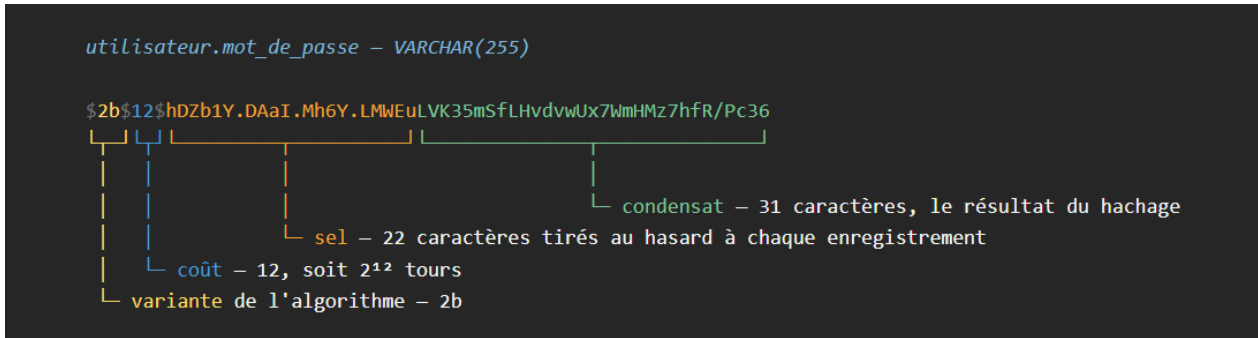


Figure 12. Les soixante caractères d'une empreinte bcrypt, champ par champ : la variante de l'algorithme, le coût, les vingt-deux caractères de sel et le condensat. Le sel n'a pas de colonne à lui : il est écrit dans l'empreinte.

Une limite de bcrypt impose une règle de plus. Le mot de passe ne doit pas dépasser 72 octets : au-delà, bcrypt tronque silencieusement, et deux mots de passe ne différant qu'après le 72^e octet ouvriraient le même compte. Le contrôle porte sur les octets et non sur les caractères, un caractère accentué en occupant deux.

À la connexion, il faut répondre sans rien trahir. Une adresse inconnue et un mot de passe faux renvoient le même message. Cela ne suffit pas : si le serveur répond instantanément quand l'adresse n'existe pas, et après le temps d'un calcul bcrypt quand elle existe, l'écart de durée désigne les comptes valides. La parade est une empreinte factice, calculée une fois au démarrage : lorsque l'adresse est inconnue, la comparaison est menée quand même, contre elle.

```
src/controllers/authController.js - la même réponse, que le compte existe ou non
11 // Comparison target when the email is unknown: constant-time response.
12 const HASH_FACTICE = bcrypt.hashSync("comparaison-a-temps-constant", COUT_BCRYPT);
13
107 // Looks up the account (no short-circuit: constant-time comparison below).
108 const utilisateur = await Utilisateur.findByEmail(email);
109
110 // Checks the password in constant time (dummy hash when the account is unknown).
111 const hashCible = utilisateur ? utilisateur.mot_de_passe : HASH_FACTICE;
112 const motDePasseValide = await bcrypt.compare(motDePasse, hashCible);
113
114 if (!utilisateur || !motDePasseValide) {
115   return res.status(401).render("connexion", {
116     erreurs: [{ msg: "Email ou mot de passe incorrect." }],
117   });
118 }
```

Figure 13. L'empreinte factice est calculée une fois au démarrage, puis sert de cible quand l'adresse est inconnue : la comparaison bcrypt a lieu dans tous les cas, et le message renvoyé est le même.

La session est régénérée avant d'accueillir l'identité. C'est une précaution contre la fixation de session : un attaquant qui serait parvenu à imposer un identifiant de session à sa victime avant qu'elle se connecte se retrouve avec un identifiant périmé. L'authentification repose ensuite sur une session serveur, pas sur un token confié au navigateur : le cookie ne porte qu'un identifiant signé, les données vivent dans une table de la base. Le client ne détient donc jamais son propre rôle, qu'il pourrait modifier.

Réglage du cookie	Valeur	Ce qu'il empêche
httpOnly	activé	Le cookie est invisible depuis JavaScript : un script injecté ne peut pas le voler.

sameSite	lax	Le cookie n'accompagne pas une requête déclenchée depuis un autre site.
secure	en production	Le cookie ne circule que sur HTTPS, jamais en clair.
maxAge	24 heures	Une session oubliée finit par expirer d'elle-même.
saveUninitialized	désactivé	Un visiteur qui ne fait que lire ne se voit attribuer aucune session : donnée non collectée, exigence de minimisation.

Pendant la session, l'utilisateur est rechargé à chaque requête. Il n'est pas lu dans la session : un compte désactivé ou anonymisé se trouve déconnecté dès la requête suivante, sans attendre l'expiration. Le coût est une lecture de plus par requête, le gain est qu'une exclusion prend effet tout de suite. La déconnexion, elle, détruit la session côté serveur : le cookie ne vaut plus rien, même copié. Sinon, elle s'éteint d'elle-même au bout de vingt-quatre heures.

En cas d'oubli, un token remplace le mot de passe le temps d'un lien. Trente-deux octets tirés au hasard partent dans le courriel ; la base n'en conserve que l'empreinte SHA-256. Une fuite de cette table ne livrerait donc aucun lien utilisable. Le token vaut trente minutes, ne sert qu'une fois, et toute nouvelle demande invalide la précédente : un seul lien valide à la fois. La même mécanique confirme une inscription, avec une validité de douze heures. Une tâche planifiée supprime chaque nuit les tokens expirés.

10.2 Protection CSRF

Une requête peut être authentique sans être voulue. Un candidat s'est connecté, puis a laissé l'onglet ouvert. Sur un autre site, une page fabrique un formulaire dirigé vers MonPremierPoste et le soumet d'elle-même ; le navigateur, voyant la destination à laquelle appartient le cookie de session, le joint. Le serveur reçoit une requête correctement authentifiée, que rien ne distingue d'une action volontaire. C'est ce qu'on appelle une falsification de requête inter-site, CSRF, pour cross-site request forgery. Rien pourtant n'a été volé, ni le cookie ni le mot de passe : c'est le navigateur du candidat qui a servi d'intermédiaire, à son insu. L'enjeu est réel : changer l'adresse électronique du profil détournerait ensuite tous les liens de réinitialisation, et donc le compte entier.

La parade tient à une asymétrie du navigateur. Le cookie porte déjà une première défense, décrite en 10.1, mais elle est confiée au navigateur et le serveur ne peut pas en constater l'effet ; il en fallait une seconde. Un site tiers peut déclencher une requête vers MonPremierPoste, il ne peut pas lire la page qu'il vise : la politique d'origine du navigateur le lui interdit. Il suffit donc d'exiger, dans toute requête modifiante, une valeur qu'on ne peut obtenir qu'en ayant lu la page. C'est le patron du token synchronisé, fourni par **csrf-sync** ; le paquet historique d'Express, **csrf**, rendait le même service mais n'est plus maintenu. Le serveur tire un token au hasard, le conserve dans la session, l'inscrit dans le formulaire, et compare les deux à la réception. Le token lui revient par un champ caché, ou par un en-tête pour la seule requête que le site émet en JavaScript.

La vérification est globale, avec une seule exception. Montée avant les routes, elle couvre toute méthode modifiante : une route écrite plus tard est protégée sans qu'on y pense, et l'oubli change de camp : il faut désormais déclarer une exception, non une protection. Il en existe une : l'envoi d'un CV arrive dans un format que l'analyseur placé en amont ne lit pas, si bien que le token, présent, y est encore illisible. Le contrôle n'est pas supprimé, il est déplacé dans la route, après réception du fichier.

Le token n'est produit que lorsqu'un formulaire va être soumis. Le conserver oblige à ouvrir une session, alors que la configuration exposée en 10.1 refuse les sessions vides, pour qu'un visiteur qui se contente de lire ne laisse aucune ligne en base. Se défendre et ne rien conserver d'inutile se contredisent donc ici : un token sur chaque page aurait créé une session à chaque offre consultée. Il n'est exposé que si une session existe déjà, et forcé sur les seules pages qui présentent un formulaire à un visiteur non connecté.

Quatre tests fixent le comportement. Sans token, avec un token inventé, ou avec un token valide mais émis pour une autre session, la requête est refusée : le token ne vaut que dans la session qui l'a reçu. Le quatrième vérifie qu'un token légitime laisse bien passer la mise à jour.

10.3 Injection SQL

Ce qu'un utilisateur tape peut devenir une instruction. Quand un visiteur retire une offre de ses favoris, le serveur écrit une requête de suppression où figurent son identifiant et celui de l'offre, un couple qui ne désigne qu'une seule ligne. Or cet identifiant n'est pas choisi par le serveur : il est inscrit dans l'adresse que le navigateur appelle, `/favoris/340`, et rien n'empêche un visiteur d'y écrire ce qu'il veut. S'il était recopié tel quel dans le texte de la requête, il suffirait d'y taper `340 OR 1=1` : la condition « cette ligne-là » devient « cette ligne-là, ou bien n'importe laquelle », et la suppression emporte la table entière, les favoris de tous les autres utilisateurs compris. Rien n'a été forcé, aucun mot de passe deviné, aucun caractère interdit employé : la saisie a simplement changé la question posée à la base. C'est ce qu'on appelle une injection SQL, une valeur lue comme une instruction.

La parade consiste à séparer les deux. La valeur n'est plus concaténée au texte de la requête. Celle-ci part seule vers le serveur de base de données, un point d'interrogation tenant la place de chaque valeur : **WHERE id_offre = ?**. Le serveur l'analyse et en fige la structure : il sait déjà quelle table il vise et sur quelles colonnes il filtre, et cela ne changera plus. Les valeurs ne sont envoyées qu'ensuite, par un second échange, et arrivent donc dans une requête dont le sens est déjà arrêté. Il n'y a plus rien à réinterpréter : l'injection ne devient pas difficile, elle devient sans objet. Les soixante-cinq requêtes du projet sont écrites ainsi.

```
src/models/Favori.js
12 // Removes the posting from the user's favorites, scoped to id_utilisateur. True if removed.
13 async function remove(idUtilisateur, idOffre) {
14   const [result] = await pool.execute(`DELETE FROM favori WHERE id_utilisateur = ? AND id_offre = ?`, [
15     idUtilisateur,
16     idOffre,
17   ]);
18   return result.affectedRows > 0;
19 }
```

Figure 14. La suppression d'un favori, dans le modèle : la requête ne porte que des emplacements, et les deux valeurs sont transmises dans un tableau à part.

10.4 XSS et échappement des sorties

Le contenu affiché n'a pas été écrit par moi. Les intitulés, les descriptions et les noms d'entreprise arrivent de trois interfaces distantes, et s'y ajoute ce que les utilisateurs saisissent. Rien de cela n'est du contenu de confiance : recopiée telle quelle dans une page, une de ces valeurs verrait ses chevrons lus comme du balisage, et le navigateur exécuterait ce qu'elle contient, une injection de contenu dans la page, ou **XSS**, pour cross-site scripting. **Le code agit alors depuis la page elle-même, et les deux protections précédentes n'y peuvent rien** : le script injecté n'a pas besoin de voler le cookie de session, qui lui reste illisible : le navigateur le joint de lui-même à toute requête partie de la page. Et le token anti-CSRF, que la page expose pour ses propres scripts, se lit exactement de la même façon. **La protection ne se joue donc pas à la saisie, mais au moment du rendu.**

Le moteur de vues échappe par défaut. EJS convertit en entités HTML toute valeur insérée par sa syntaxe courante, `<%= %>` : les chevrons d'un titre hostile cessent d'ouvrir une balise, et la valeur s'affiche en toutes lettres au lieu d'agir. La syntaxe `<%- %>`, elle, **n'échappe rien** : elle écrit la valeur telle quelle. Le projet ne s'en sert que sur du HTML qu'il a lui-même écrit, comme l'inclusion d'un gabarit dans un autre. **Une donnée venue d'un utilisateur ou d'une API n'y passe jamais.**

L'e-mail sort du moteur de vues. Une candidature part par courrier électronique, dans un message que l'application signe de son propre domaine, et ce message n'est pas une vue : son HTML est assemblé à la main. Ni EJS ni la politique de sécurité du contenu (qui interdit au navigateur d'exécuter un script inséré dans la page, et dont la 10.7 détaille les réglages) n'ont cours sur ce trajet : un logiciel de messagerie ne lit aucun en-tête HTTP. **L'échappement a donc dû être écrit.** Une fonction convertit les caractères qui ont un sens en HTML avant chaque insertion. **Le titre de l'annonce y passe comme le reste** : celui-là même qui vient d'une source, et qui n'a cette fois traversé aucune vue. Jusqu'à son adresse, placée en attribut `href`, où un simple guillemet refermerait l'adresse, et le lien ne mènerait plus où il annonce.

```
src/services/emailService.js
5 // Escapes HTML special characters of user data before injecting it into the email (the "&" first).
6 function echapperHtml(valeur) {
7   return String(valeur)
8     .replaceAll("&", "&amp;")
9     .replaceAll("<", "&lt;")
10    .replaceAll(">", "&gt;")
11    .replaceAll("'", "&quot;")
12    .replaceAll('"', "&#39;");
13 }

Titre reçu d'une source, tel qu'il arrive en base
<img src=x onerror=alert(1)>

La ligne produite dans l'e-mail du recruteur
<p>Annonce concernée : <a href="https://exemple.fr/offre/340">&lt;img src=x onerror=alert(1)&gt;</a></p>
```

└ encodé : la balise se lit, elle n'agit pas

Figure 15. La fonction d'échappement du service d'envoi, et ce qu'elle produit sur le titre piégé d'une annonce reçue d'une source : la balise est encodée, le destinataire la lit au lieu de la subir.

10.5 Contrôle d'accès et cloisonnement des documents

Un droit se vérifie à chaque requête, jamais à l'affichage. Masquer un bouton ne protège rien : l'adresse reste appellable. Deux gardes encadrent les routes (l'une exige une session ouverte, l'autre le rôle d'administrateur), et toute requête portant sur une ressource personnelle est filtrée par l'identifiant de son propriétaire, et pas seulement par celui de la ressource. La nuance est le sujet même : demander « le document numéro 12 » puis vérifier à qui il appartient est une occasion d'oubli ; demander « le document numéro 12 appartenant à l'utilisateur courant » ne peut pas se tromper. Une demande qui n'aboutit pas reçoit un **404, jamais un 403** : un 403 confirmerait que la ressource existe.

Et pourtant ce contrôle ne protégeait pas les fichiers. Les CV et les lettres de motivation ne sont pas stockés par l'application : ils vivent chez un prestataire, et c'est le premier intérêt du choix : le fichier ne touche jamais le disque du serveur. Il est reçu en mémoire et part directement au stockage, sans jamais atterrir à côté des fichiers que l'application publie, où une erreur de configuration suffirait à le rendre lisible. À l'entrée, seul un PDF est accepté, type déclaré et extension à l'appui, un filtre utile, mais qui ne prouve rien : ces deux informations viennent du client. Restait à savoir qui, chez le prestataire, pouvait le lire. Dans ma première version, la vue affichait l'adresse que ce prestataire renvoyait. Le contrôle d'accès était correct (un

utilisateur ne voyait que ses propres documents), mais cette adresse était publique, et sa forme parlait d'elle-même : `.../documents/user_7_cv.pdf`. Le nombre est l'identifiant du propriétaire ; en écrire un autre donnait le CV d'un autre candidat, sans même être connecté : le prestataire ne connaît ni mes sessions ni mes comptes, il sert le fichier à qui possède l'adresse. **Mon application contrôlait qui obtenait l'adresse, pas qui obtenait le fichier.**

La correction tient en deux étages, et aucun ne suffisait seul. Les documents sont désormais déposés en livraison restreinte : le prestataire refuse toute adresse non signée. Et le navigateur ne lui parle plus : le serveur récupère le fichier, après avoir vérifié que le demandeur en est le propriétaire, puis le retransmet lui-même, dans un format que le serveur impose et n'hérite jamais du fichier déposé. **La faille ne venait pas d'une ligne fausse, mais d'une hypothèse : j'avais cru qu'une adresse difficile à deviner était une adresse protégée.**

```
src/controllers/compteController.js
163 // --- Document relay ---
164 // The two routes below serve the file from the server instead of letting the view publish a
165 // Cloudinary URL: access goes through findByIdAndUser, and the browser contacts no third party.
166 async function servirDocument(req, res, next, choisirUrl) {
167   const id = Number.parseInt(req.params.id, 10);
168   const document = Number.isInteger(id) && id >= 1 ? await Document.findByIdAndUser(id, req.user.id_utilisateur) : null;
169   // 404 and not 403: a 403 would confirm the document exists and allow account enumeration.
170   if (!document) return next();
171
172   const { contenu, typeMime } = await recupererFichier(choisirUrl(document));
173   // A CV must never sit in a shared cache: "private" forbids proxies, "no-store" the client disk.
174   res.set("Cache-Control", "private, no-store");
175   res.type(typeMime).send(contenu);
176 }
```

Figure 16. Le relais serveur : une seule fonction porte le filtrage par propriétaire, le choix du 404 et l'interdiction de mise en cache.

10.6 Rate limiting

Une protection correcte n'empêche pas la répétition. Un mot de passe bien haché ne sert à rien si l'on peut en essayer des milliers, et un formulaire qui déclenche un courriel peut servir à en inonder un tiers. Les compteurs sont tenus par `express-rate-limit`, qui écrit aussi les en-têtes normalisés annonçant le plafond et ce qu'il en reste. Le filet général est monté **avant l'analyse du corps de la requête et avant la session** : une fois le plafond atteint, la requête est refusée sans avoir rien coûté, ni lecture de session, ni requête en base. Cinq limiteurs sont en place, chacun calibré sur ce qu'il protège.

Limiteur	Plafond	Ce qu'il protège
Général	300 requêtes par minute	Le martèlement automatisé, quelle que soit la route.
Connexion	5 échecs par quart d'heure	La force brute. Seuls les échecs sont comptés : une journée de travail normale n'est jamais bloquée.
Inscription	5 par heure	L'envoi massif de courriels vers des adresses non vérifiées.
Mot de passe oublié	5 par heure	Même risque, même plafond.
Candidature et relance	20 par heure et par compte	L'envoi en masse depuis un compte légitime.

Le limiteur des candidatures compte par utilisateur, et non par adresse réseau : chaque candidat n'est mesuré que sur ses propres envois, qu'il partage ou non sa connexion avec d'autres. Un compteur unique couvre les candidatures et les relances.

Derrière un proxy inverse, l'adresse du client n'est plus celle de la connexion. Le serveur ne voit que celle du proxy ; la vraie arrive dans un en-tête, **X-Forwarded-For**. L'ignorer en production, c'est compter tous les visiteurs sous une seule adresse : cinq échecs de connexion, et le site est bloqué pour tout le monde. Le croire là où aucun proxy ne le pose, c'est faire confiance à une valeur que le premier venu écrit lui-même, et tous les plafonds tombent. L'application déclare donc **combien de proxys se trouvent devant elle**, par variable d'environnement : un en production, aucun en développement.

10.7 En-têtes de sécurité

Un en-tête de sécurité est une consigne donnée au navigateur. Une fois la page partie, c'est lui qui décide ce qu'il exécute, ce qu'il affiche et ce qu'il transmet. Ces consignes se comptent par dizaines, chacune avec sa syntaxe, et les valeurs recommandées changent avec les navigateurs : les écrire à la main reviendrait à assurer cette veille soi-même. Le paquet **helmet** en fournit un jeu par défaut, tenu à jour, qu'il ne reste qu'à resserrer. Il est monté avant les fichiers statiques, de sorte que les consignes accompagnent toutes les réponses.

Ce que ces consignes empêchent. Le site refuse d'être affiché dans le cadre d'un autre : c'est la parade au clickjacking, où une page invisible est superposée à un leurre pour faire cliquer l'utilisateur sur ce qu'il ne voit pas. Les formulaires ne peuvent être soumis qu'au site lui-même. Le navigateur n'exécute aucun gestionnaire d'événement écrit en attribut : la parade au **onerror=** inséré dans une page resterait inerte. Enfin, il ne va chercher styles, polices et images que sur l'origine du site : trois directives resserrées, la politique de sécurité du contenu livrée par helmet acceptant par défaut les styles écrits en ligne et n'importe quel domaine servi en HTTPS, dont le projet n'a aucun besoin puisqu'il héberge ses propres polices. **Ces en-têtes ne remplacent aucune des mesures précédentes : ils limitent les dégâts le jour où l'une d'elles serait prise en défaut.**

```

    src/app.js
30 // --- HTTP security headers ---
31 // Mounted first, before the static files: the headers must travel with EVERY response.
32 // CSP stricter than Helmet's defaults: no inline script or style, own fonts, all 'self'.
33 app.use(
34   helmet({
35     contentSecurityPolicy: {
36       useDefaults: true,
37       directives: {
38         defaultSrc: ["'self'"],
39         scriptSrc: ["'self'"],
40         // Helmet allows 'unsafe-inline' and https: for styles by default. Not needed here.
41         styleSrc: ["'self'"],
42         // Fonts self-hosted in public/fonts: no third-party domain to allow.
43         fontSrc: ["'self'"],
44         // No third-party opening: CV previews go through the server, not a Cloudinary URL.
45         imgSrc: ["'self'"],
46         objectSrc: ["'none'"],
47       },
48     },
49   })
50 );

```

Figure 17. La configuration des en-têtes : en vert les trois directives resserrées, et les commentaires qui disent pourquoi le projet n'a besoin d'aucun domaine tiers.

10.8 RGPD

Ne pas collecter est la première protection. L'inscription ne demande que le nom, le prénom et l'adresse électronique. Aucun traceur, aucun service tiers appelé par le navigateur, un seul cookie, celui de la session. La politique de confidentialité indique pour chaque traitement ce qui l'autorise : l'exécution du service, la demande explicite du candidat, l'intérêt légitime à protéger les comptes. **Le consentement n'est donc pas la base légale du compte** : rien n'est à renouveler périodiquement. Ce qui reste dû, c'est la limitation de la durée, et aucune purge ne vise encore les comptes inactifs. La règle retenue, **anonymiser deux ans après la dernière connexion**, après deux relances par courriel, figure parmi les évolutions.

Ce que l'application détient, elle sait le montrer. Une page récapitule, pour l'utilisateur connecté, son compte, ses documents, ses favoris et ses candidatures, énumérés et non simplement comptés, car pour un favori la donnée personnelle est le **lien** entre l'utilisateur et l'offre, pas l'offre elle-même.

L'effacement se fait par anonymisation. Une suppression pure et simple casserait l'intégrité référentielle de la base : la ligne survit donc, mais vidée de tout ce qui identifie : nom et prénom neutralisés, adresse électronique remplacée par une adresse non délivrable, empreinte du mot de passe vidée, compte désactivé et session détruite sur-le-champ. Les documents, eux, sont supprimés : **du prestataire d'abord, de la base ensuite**. Si cet échange échoue, l'anonymisation n'a pas encore eu lieu et l'opération reste reprenable ; l'ordre inverse laisserait chez le prestataire des fichiers que plus rien ne relierait à un compte. L'action étant irréversible, le mot de passe est redemandé sur l'écran de confirmation.

>_ 11. Tests et jeu d'essai

Trois cent cinquante-huit tests, répartis en quinze fichiers. Quatorze suites unitaires et une suite d'intégration, exécutées par une seule commande, en une vingtaine de secondes. Cette section explique ce qui est vérifié, ce qui ne l'est pas, et pourquoi la frontière a été tracée là.

11.1 Stratégie de tests

On ne teste pas tout : on teste ce qui peut casser en silence. Une erreur d'affichage se voit tout de suite, elle n'a pas besoin d'un test pour être trouvée. En revanche, si un mot-clé mal délimité fait passer une offre « senior » pour une offre junior, rien ne le signale : il faudrait relire des centaines d'annonces pour s'en apercevoir. Ce sont ces défauts-là, invisibles à l'écran, que les tests surveillent.

Ce qui est testé
Normalisation des trois sources : champs, salaires, régions, contrats.
Détection junior et pertinence : les frontières des mots-clés.
Validateurs de formulaire, cas passants et cas refusés.
Contrôle d'accès, CSRF, échappement, plafonds de requêtes.
Une seule synchronisation à la fois par source, y compris après un échec.
Expiration suspendue tant qu'une collecte n'a pas enregistré la moitié du catalogue.

11.2 Tests unitaires

Les fonctions qui décident n'ont pas d'apparence. Cette offre s'adresse-t-elle à un junior ? Cette annonce est-elle pertinente ? Ce mot de passe respecte-t-il la politique ? Rien de tout cela ne se vérifie à l'œil : il faut poser la question à la fonction et comparer sa réponse à celle qu'on attend. C'est exactement ce qu'est un test unitaire : une entrée, une sortie attendue, **sans base, sans serveur et sans réseau**.

Les cas qui comptent sont les frontières. La détection junior repose sur des mots-clés cernés par des délimiteurs, et les tests vérifient précisément ce que ces délimiteurs protègent : que « **expert** » **ne se déclenche pas dans « expertise »**, que « **lead** » ne se déclenche pas dans « **leader** », qu'un intitulé écrit sans accents est reconnu comme un intitulé accentué. Ce sont des cas qu'on n'imagine pas en écrivant la fonction, et plusieurs ont révélé un défaut réel au moment où j'écrivais le test.

```

tests/unit/junior.test.js
112 describe("Normalisation du texte analysé", () => {
113   test("les accents et la casse sont ignorés", () => {
114     expect(estJunior({ titre: "DÉVELOPPEUR DÉBUTANT" })).toBe(true);
115   });
116
117   test("le pluriel et le féminin sont couverts par le suffixe optionnel", () => {
118     expect(estJunior({ titre: "Développeur", description: "Ouvert aux débutantes." })).toBe(true);
119   });
120
121   test("les bornes de mot évitent le faux positif « expert » dans « expertise »", () => {
122     // Without the bounds, "expertise" would trigger the senior veto and the posting be lost.
123     expect(estJunior({ titre: "Développeur junior", description: "Vous monterez en expertise." })).toBe(true);
124   });
125 });

$ npm test
Test Suites: 15 passed, 15 total
Tests:       358 passed, 358 total
Time:        16.6 s

```

Figure 18. Trois tests réels de la détection junior : une offre en entrée, le résultat attendu en sortie ; le dernier vérifie que « expert » ne se déclenche pas dans « expertise ». En bas, ce que donne la suite entière, tests d'intégration compris.

Jest fait à lui seul les trois choses dont un test a besoin. Il lance les tests et signale ceux qui échouent ; il fournit la façon d'écrire « je m'attends à ce que le résultat soit celui-ci », le **toBe(true)** de la figure ; et il permet de remplacer une dépendance par un faux, pour vérifier une fonction sans envoyer de vrai courriel. Ailleurs, ces trois besoins demandent trois paquets à réunir. Deux réglages l'accompagnent : **cross-env**, parce que la façon de fixer une variable d'environnement avant une commande dépend du système, et que la même commande doit fonctionner sur mon poste comme sur le serveur qui rejoue les tests ; et l'exécution **un fichier après l'autre**, parce que les tests d'intégration partagent une même base et se videraient les tables les uns sous les autres.

Le résultat : trois cent trente-quatre tests unitaires, dans quatorze fichiers. Ils s'exécutent en une quinzaine de secondes et couvrent la mise en forme des offres venues des trois sources, le tri des annonces hors-cible, la détection des offres juniors, la validation des formulaires, les appels aux interfaces distantes et les garde-fous de la synchronisation.

11.3 Tests d'intégration

Une seconde suite teste l'application entière, pas une fonction isolée. Elle envoie de vraies requêtes, comme le ferait un navigateur : chacune passe par les routes, les middlewares, le contrôleur, la base et le gabarit de page. Ces tests travaillent sur une **base séparée**, construite à partir du même fichier de structure que la production, et s'y connectent avec le **même compte que l'application**, celui qui ne peut que lire, insérer, modifier et supprimer. D'où un détail révélateur : pour vider les tables entre deux essais, ils utilisent **DELETE** et non **TRUNCATE**, plus rapide mais réservé aux comptes qui ont davantage de droits. C'est plus lent, et c'est voulu : **si les tests tournaient avec plus de droits que l'application, ils ne testeraient pas l'application.**

11.4 Tests de sécurité

Une protection retirée par inadvertance ne casse rien de visible. L'application continue de fonctionner, simplement elle ne protège plus. C'est le cas type où seul un test avertit. Cinq familles de vérifications l'encadrent.

- Chaque route protégée redirige un visiteur sans session vers la page de connexion.
- Un candidat ne peut pas atteindre l'administration, ni lire les données d'un autre candidat : la vérification porte sur des ressources appartenant à un second compte.
- Une requête sans token CSRF, ou avec un token d'une autre session, est refusée.
- Une chaîne contenant des balises, saisie dans un champ, ressort échappée dans la page.
- Les plafonds de requêtes se déclenchent au bon rang.

11.5 Jeu d'essai

Un jeu de données pensé pour couvrir les cas, pas pour remplir l'écran. Un script remplit la base de deux comptes (un candidat et un administrateur) et de douze offres, avec leurs favoris et leurs candidatures ; il refuse de s'exécuter si l'environnement est la production. Ces douze offres ne se ressemblent pas : elles couvrent les six types de contrat, un salaire annuel, mensuel, horaire ou absent, le télétravail et le travail sur site, les offres ouvertes aux juniors et les autres, une offre expirée et une offre masquée par la modération.

Chaque variante déclenche un affichage différent. Un salaire absent ne doit pas produire « null € », une offre masquée ne doit apparaître nulle part, une offre expirée doit rester visible dans les candidatures déjà envoyées mais disparaître de la recherche. Sans cette variété, ces cas ne seraient rencontrés qu'une fois le site en ligne, **c'est-à-dire par un utilisateur.**

>_ PARTIE 5 • MISE EN PRODUCTION

>_ 12. Déploiement et exploitation

Le site est en ligne, et il se met à jour tout seul. Un envoi sur la branche principale déclenche l'analyse du code, la suite de tests complète sur une base neuve, puis (et seulement si tout est vert) la reconstruction et le redémarrage sur le serveur. Cette section décrit les environnements, la mise en production, la chaîne automatisée, les sauvegardes, et ce qui permet de savoir que l'application va bien.

12.1 Environnements

Trois environnements, une seule source pour le schéma. `sql/schema.sql` est joué à l'identique dans les trois cas : par un script en local, par l'image MySQL au premier démarrage d'un conteneur, par l'intégration continue avant les tests. Un schéma recopié à la main quelque part aurait divergé dès la première modification. Ce qui diffère tient en quelques variables : le mode d'exécution, l'adresse publique, le nombre de proxys, les identifiants des services externes ; le code, lui, est strictement le même.

Environnement	Où	Base de données
Développement	Poste de travail, ou conteneurs en local	monpremierposte, jeu d'essai chargé
Test	Poste de travail et intégration continue	monpremierposte_test, vidée avant chaque test
Production	Serveur privé virtuel, conteneurs	monpremierposte, née vide et remplie par les collectes

La production ne reçoit jamais de copie de la base de développement. Elle apporterait les comptes de démonstration et les offres factices. Le catalogue se remplit tout seul à la première collecte.

12.2 Comptes et droits d'accès à la base

Deux comptes, deux métiers. Le compte administrateur crée la structure ; le compte applicatif manipule les données. L'application n'utilise jamais le premier.

Compte	Droits	Utilisé par
Administrateur	Tous, sur le serveur de base de données	La création du schéma et des comptes, à l'installation.
Compte applicatif	Lire, insérer, modifier, supprimer, sur la seule base du projet	L'application, en fonctionnement normal.

Le compte applicatif ne peut pas modifier la structure. Il n'a le droit ni de créer, ni de supprimer, ni de modifier une table. Une injection SQL qui parviendrait malgré tout à s'exécuter ne pourrait donc pas effacer le catalogue : elle se heurterait au refus du serveur de base de données. C'est une seconde ligne de défense, derrière les requêtes préparées de 10.3, et **elle tient même si la première cède.**

12.3 Mise en production : VPS, Docker Compose et Caddy

Le site tourne sur un serveur loué, que j'administre entièrement. C'est ce qu'on appelle un serveur privé virtuel, ou VPS : une machine louée chez un hébergeur, sur laquelle on installe ce que l'on veut, à la différence d'un hébergement mutualisé, où l'on dépose seulement des fichiers dans un environnement déjà tout fait.

Sur cette machine, rien n'est installé directement : tout passe par des **conteneurs**, des paquets qui embarquent un programme et tout ce dont il a besoin pour fonctionner.

Docker est l'outil qui fabrique et lance ces conteneurs. On lui décrit, dans un fichier, tout ce dont le projet a besoin : la version de Node, le serveur MySQL, le schéma à créer, le compte aux droits restreints, la quinzaine de variables d'environnement. **Écrire tout cela dans une notice, c'est espérer que celui qui installe n'oublie rien ; l'écrire dans un fichier que Docker exécute, c'est le rendre reproductible.** Le bénéfice se voit au déploiement : **le conteneur validé sur mon poste est exactement celui qui tourne en production**, sans différence possible de version ni de réglage.

Trois conteneurs : l'application, sa base, et Caddy. Ce dernier est un proxy inverse, c'est-à-dire un serveur placé devant l'application : il reçoit toutes les requêtes du public et les lui transmet. Il se charge aussi du chiffrement, et c'est ce qui m'a décidé : **il obtient le certificat HTTPS et le renouvelle seul**, sans qu'aucune date ni aucun fichier de certificat n'apparaisse dans sa configuration. Quant au conteneur de l'application, il ne s'exécute pas en administrateur : une faille dans le code ne donnerait pas la main sur le conteneur.

```
$ docker compose ps
SERVICE      IMAGE                STATUS              PORTS
app            mon-premier-poste-app Up 9 days (healthy) 3000/tcp
caddy          caddy:2              Up 11 days          0.0.0.0:80->80/tcp, [::]:80->80/tcp,
                                0.0.0.0:443->443/tcp, [::]:443->443/tcp,
                                443/udp, 2019/tcp
db             mysql:8              Up 10 days (healthy) 3306/tcp, 33060/tcp

$ curl https://monpremierposte.fr/sante
{"statut":"ok"}

$ openssl s_client -connect monpremierposte.fr:443 | openssl x509 -noout -issuer -dates
issuer=C=US, O=Let's Encrypt, CN=YE1
notBefore=Aug 24 13:42:32 2026 GMT
notAfter=Nov 22 13:42:31 2026 GMT
```

Figure 19. Trois relevés du 4 septembre 2026, sur le serveur et depuis l'extérieur : les conteneurs, leur état de santé et leurs ports (seul le proxy en publie vers l'extérieur) ; la réponse de la route de santé ; le certificat, émis le jour de la mise en ligne et renouvelé automatiquement tous les trois mois.

Un seul conteneur est joignable depuis l'extérieur. Le proxy. L'application et la base se parlent sur un réseau privé interne, et le port de la base n'est jamais ouvert vers l'extérieur. Ce point m'a demandé de comprendre un piège : **un pare-feu ne suffirait pas ici**, parce que Docker écrit ses propres règles de filtrage, examinées avant celles du pare-feu. Un port ouvert par Docker reste donc joignable alors même que le pare-feu affirme le contraire. Ce qui ferme réellement la porte, c'est de ne pas ouvrir le port.

La base naît vide, et le premier administrateur est promu à la main. Le schéma et les droits ne sont appliqués qu'une fois, au tout premier démarrage. Ensuite, tout compte créé reçoit le rôle candidat : il n'existe aucune règle du type « le premier inscrit devient administrateur », qui serait, sur un site fraîchement publié, une course que n'importe qui pourrait gagner.

12.4 Intégration et déploiement continu avec GitHub Actions

Le site est en ligne ; reste à le mettre à jour sans ouvrir une session sur le serveur à chaque correction. La première mise en ligne a demandé une série de gestes qui ne se répètent jamais : installer Docker, écrire la configuration, laisser la base naître vide, promouvoir le premier administrateur. Tout le reste, en revanche, se répète et tient en trois commandes : **git pull --ff-only** pour tirer le code, **docker compose up --build --wait**

pour reconstruire l'image et attendre que les conteneurs se déclarent sains, **docker image prune** pour effacer celles que les constructions précédentes ont laissées. **Ces trois commandes, GitHub Actions les exécute à ma place**, avec une clé qui ne peut rien lancer d'autre sur le serveur.

L'intégration continue, c'est vérifier automatiquement chaque envoi de code. De la fusion d'une pull request à la mise en ligne, la chaîne compte six étapes :

- Je fusionne une pull request dans la branche principale. C'est mon seul geste.
- GitHub Actions démarre une machine neuve, installe les dépendances à la version exacte du verrou du dépôt, applique le schéma sur une base vide, vérifie le style du code et joue la suite de tests.
- Le déploiement n'est atteint que si cette vérification est verte, et uniquement sur la branche principale. Les tests rejouent sur le commit de fusion : ce qui part en ligne est exactement ce qui a été vérifié.
- La chaîne se connecte au serveur en SSH, sans passer aucune commande. La clé privée et les empreintes des serveurs connus voyagent par des variables d'environnement, jamais dans le texte de la commande, où une valeur interpolée deviendrait une partie du script exécuté.
- Sur le serveur, cette clé porte une commande forcée : la seule chose qu'elle peut déclencher est le script de déploiement. Une clé dérobée n'ouvrirait aucun interpréteur de commandes.
- Le script exécute alors ces trois commandes, dans cet ordre.

Le chemin inverse est verrouillé lui aussi. Le dépôt est privé : pour pouvoir le rafraîchir, le serveur s'y authentifie avec une clé de déploiement, **en lecture seule et limitée à ce seul dépôt**. Les deux liens sont donc étroits, chacun dans son sens : la chaîne ne peut que déclencher un déploiement, le serveur ne peut que lire le code.

Le script tient en une vingtaine de lignes. Elles reprennent, sans en changer une seule, les commandes que je tapais sur le serveur, et la moitié d'entre elles explique pourquoi chacune est écrite ainsi.

```
scripts/dploiement.sh
1  #!/bin/bash
2  # Deploys the branch checked out on the server: refreshes the code, rebuilds the image, restarts
3  # the containers. Invoked by the continuous integration over SSH, as a forced command.
4  #
5  # The whole body sits in a brace group: bash parses it entirely before running it, so the
6  # "git pull" below may rewrite this very file without the running shell reading a moved offset.
7  {
8      set -euo pipefail
9
10     cd "$(dirname "$0")/.."
11
12     # --ff-only refuses to deploy a diverged tree instead of merging blindly on the server.
13     git pull --ff-only
14
15     # The code is not mounted into the container: without --build the previous image keeps running.
16     docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build --wait
17
18     # Images left behind by the previous builds, which would fill the disk over time.
19     docker image prune -f >/dev/null
20
21     echo "Déployé : $(git rev-parse --short HEAD) - $(git log -1 --pretty=%s)"
22
23     exit 0
24 }
```

Figure 20. Le script de déploiement en entier. Le drapeau de reconstruction est celui dont l'absence resservirait la version précédente ; le commentaire le précède est là pour qu'on ne le retire pas un jour par économie.

12.5 Sauvegarde et restauration

Une sauvegarde n'est pas un fichier, c'est un retour en arrière qui marche. Tant qu'on ne l'a pas fait au moins une fois, on ne sait pas si on en a une. Le contenu de la base est exporté chaque nuit à 4 h 30 par une tâche planifiée sur la machine hôte, c'est-à-dire le VPS lui-même, en dehors des conteneurs. Cette machine, pourtant, ne sait pas parler à la base : aucun client MySQL n'y est installé, et le port de la base n'est joignable que depuis le réseau privé interne.

L'export est donc lancé à l'intérieur du conteneur de base. Trois choses se passent alors, à deux endroits :

Ce qui se passe	Où
La tâche planifiée se déclenche à 4 h 30 et lance le script	Sur le VPS
La commande d'export s'exécute et lit la base	Dans le conteneur de base
Le résultat compressé se pose sous un nom horodaté, lisible par son seul propriétaire, conservé quatorze jours	Sur le VPS

Le fichier ne reste pas dans le conteneur. Les conteneurs sont reconstruits à chaque déploiement : une sauvegarde écrite à l'intérieur de l'un d'eux serait une copie rangée dans l'objet même qu'elle protège, et effacée au déploiement suivant.

```
scripts/db-backup-prod.sh
21 # mysqldump runs inside the database container; the host has no client and port 3306 is not published.
22 # The application account is enough: --single-transaction avoids LOCK TABLES, --no-tablespaces avoids PROCESS.
23 # The password is read from the container environment, never from the command line nor from cron.
24 "${COMPOSE[@]}" exec -T db sh -c 'MYSQL_PWD="${MYSQL_PASSWORD}" exec mysqldump --user="${MYSQL_USER" \
25 --single-transaction --no-tablespaces "${MYSQL_DATABASE}" | gzip >"$temporaire"
```

Figure 21. Les trois commentaires du dépôt disent pourquoi la commande est écrite ainsi : l'outil et le mot de passe sont déjà dans le conteneur, et le compte applicatif suffit à faire le travail.

La restauration emprunte le même chemin, en sens inverse. Le fichier est décompressé sur l'hôte et reversé dans le conteneur par le même passage, cette fois avec le compte administrateur, puisqu'il recrée les tables. Elle reste volontairement manuelle : aucun script du projet ne la déclenche, car une commande qui écrase une base finit par être lancée sur la mauvaise. L'avoir exécutée pour de bon a révélé cette contrainte de compte, que la rédaction de la procédure n'aurait pas fait apparaître.

```
README.md
1062 cd $HOME/mon-premier-poste
1063 compose="docker compose -f docker-compose.yml -f docker-compose.prod.yml"
1064
1065 printf 'CREATE DATABASE restauration_test;\n' \
1066 | $compose exec -T db sh -c 'MYSQL_PWD="${MYSQL_ROOT_PASSWORD}" exec mysql -uroot'
1067
1068 gunzip -c $HOME/sauvegardes/monpremierposte_2026-08-25_11-09.sql.gz \
1069 | $compose exec -T db sh -c 'MYSQL_PWD="${MYSQL_ROOT_PASSWORD}" exec mysql -uroot restauration_test'
```

Figure 22. Les commandes de la restauration, telles qu'elles ont été exécutées sur le serveur puis consignées dans le fichier de documentation du dépôt. La base jetable est créée, puis alimentée par le même passage que l'export.

On essaie d'abord la restauration dans une base jetable. L'export est versé dans une base créée pour l'occasion, dont on compare les comptages de lignes avec la base vivante avant de la supprimer. C'est la seule façon de prouver qu'une sauvegarde est restaurable sans toucher aux données réelles.

La sauvegarde survit à une reconstruction des conteneurs, mais pas à la perte du serveur. Elle est rangée sur la machine même qu'elle protège. Il faudrait qu'une copie parte ailleurs, ce qui n'est pas en place.

12.6 Supervision et journalisation

Une route dédiée dit si l'application va bien. Elle ne se contente pas de répondre : elle interroge la base. Une application debout devant une base tombée n'est pas en bonne santé, et une sonde qui ne le verrait pas ne servirait à rien. Le conteneur s'en sert lui-même, à intervalle régulier. Un détail m'a plu dans sa mise en œuvre : l'image ne contient aucun client HTTP, et il aurait fallu en installer un pour la sonder : **Node sait le faire nativement, la sonde tient en une ligne.** Une période de grâce au démarrage évite qu'un échec initial soit compté comme une panne, et un conteneur arrêté est relancé, y compris après un redémarrage du serveur.

Un service extérieur appelle le site toutes les cinq minutes. Le conteneur lit déjà la sonde de l'intérieur, mais un surveillant installé sur la machine qu'il surveille s'éteint avec elle : il fallait un appel venu du dehors. UptimeRobot le fait depuis ses propres machines : il demande la page d'accueil à intervalle régulier et **m'envoie un courriel dès qu'elle ne répond plus.** Cinq minutes est le pas le plus court de l'offre gratuite ; interroger plus souvent supposerait un abonnement, pour un gain sans commune mesure avec ce que ce service remplace, où j'aurais découvert la panne en ouvrant le site moi-même. La page d'accueil interroge la base pour afficher les dernières offres, **donc une base tombée déclenche l'alerte elle aussi.**

Tout n'est pas consigné, et c'est un choix. L'application n'écrit aucun fichier : elle affiche ses messages sur la sortie standard, et le moteur de conteneurs les collecte. Le gestionnaire d'erreurs n'y envoie que les erreurs de serveur ; un refus attendu, comme une limite de requêtes atteinte, n'est pas un incident, et le consigner comme tel noierait les vrais. À côté, chaque tâche planifiée écrit son bilan chiffré en une ligne. Le résultat tient en peu de chose : un peu plus de deux cents lignes en dix jours, **dont six échecs, tous venus des sources externes.**

```
$ docker compose Logs app
✓ Connexion MySQL établie
✓ Serveur SMTP (Brevo) prêt
✓ Cloudinary prêt
↳ « Adzuna » planifiée (0 3 * * *).
↳ « France Travail » planifiée (0 6,12,18 * * *).
↳ « La Bonne Alternance » planifiée (0 7,13,19 * * *).
📅 Tâches planifiées activées.
[...]
🕒 Synchronisation planifiée « France Travail » : démarrage...
✓ Synchronisation « France Travail » terminée :
{"source":"France Travail","nbRecuperees":996,"nbIgnorees":55,"nbDoublons":923,"nbErreurs":0,"anomalie":null}
[...]
✗ Synchronisation « France Travail » échouée : France Travail : échec de la recherche (HTTP 429)
```

Figure 23. Le journal de l'application en production. Le démarrage vérifie ses trois dépendances, les trois sources sont planifiées avec leur horaire, et chaque synchronisation écrit son bilan. La dernière ligne est un échec de source, avec son code : la suivante rattrape, et rien d'autre ne le signale.

>_ 13. Documentation et veille

Un projet ne s'arrête pas le jour où il fonctionne. Il sera repris par quelqu'un qui n'était pas là quand il a été écrit, et il repose sur des bibliothèques qui, elles, continuent de changer. La documentation répond au premier besoin, la veille au second. Ni l'une ni l'autre n'ajoute de fonctionnalité, et c'est bien pour cela qu'elles se remettent si facilement à plus tard.

13.1 Documentation technique

Le README est une notice d'installation avant d'être une présentation. C'est le parti pris, et il vient d'une question simple : que cherche quelqu'un qui ouvre le dépôt pour la première fois ? À faire tourner le projet. La présentation du produit tient donc en quelques lignes, et l'essentiel décrit le chemin qui va du dépôt cloné à l'application qui répond.

Étape	Ce qu'elle demande
1. Cloner et installer	Le dépôt, puis les dépendances. Node 22 ou plus, MySQL 8.
2. Créer la base	Un compte administrateur du serveur de données, le temps de la création.
3. Créer le compte applicatif	Un second compte, dont les droits s'arrêtent à cette base (12.2).
4. Renseigner les variables	Un fichier d'exemple versionné liste les clés attendues.
5. Charger les données	Les sources, indispensables ; le jeu d'essai, facultatif.
6. Démarrer	L'application répond sur le port choisi.

Il est bilingue, la version anglaise en premier. Les deux sont complètes et repliables. Rien ne l'imposait : le site est francophone, et une notice en français aurait servi tous ceux à qui il s'adresse. Mais un dépôt public ne se lit pas seulement là où il a été écrit, et une documentation qui n'existe qu'en français ferme le projet à qui voudrait l'installer sans parler la langue. Les deux versions se maintiennent donc ensemble, à chaque modification de la notice.

13.2 Veille technologique et de sécurité

Personne ne prévient qu'une dépendance a vieilli. Une faille déclarée, si : une alerte tombe le jour même. Mais la fin du support d'une version, un format d'interface qui change, une condition d'utilisation modifiée, rien ne les signale. La veille est le dispositif qui comble cet écart.

Source	Ce qu'elle apporte	Rythme
Alertes de sécurité du dépôt <i>github.com/advisories</i>	Une notification quand une dépendance, directe ou indirecte, est déclarée vulnérable.	Automatique
Node.js Security Releases <i>nodejs.org/en/blog/vulnerability</i>	Les publications de sécurité de Node, annoncées avant leur sortie.	Sur inscription
Calendrier de publication de Node <i>nodejs.org/en/about/previous-releases</i>	Les dates de fin de support actif et de fin des correctifs de sécurité.	À chaque montée de version
CERT-FR (ANSSI) <i>cert.ssi.gouv.fr</i>	Les vulnérabilités marquantes et les recommandations de durcissement.	Hebdomadaire
Documentation des trois interfaces d'offres	Les évolutions de format et les changements de conditions d'utilisation.	Ponctuel

Un exemple : je sais jusqu'à quand mon socle est couvert. Le projet tourne sur **Node 22**, version déclarée à trois endroits qui doivent s'accorder : le fichier de dépendances, la chaîne d'intégration continue et l'image du conteneur. Le calendrier officiel de Node dit ce que cette version vaut aujourd'hui : son support actif s'est achevé le 21 octobre 2025, et elle reçoit des correctifs de sécurité **jusqu'au 30 avril 2027**.

Rien dans le projet ne signale cette date : il faut être allé la chercher. Node 20, la version précédente, n'est plus corrigée depuis le 30 avril 2026, et une application restée dessus continue de fonctionner sans que rien ne l'avertisse. Connaître l'échéance permet de préparer la montée vers une version encore suivie, au lieu de la découvrir le jour où une faille sort.

>_ PARTIE 6 • RETOUR SUR LE PROJET

>_ 14. Bilan et perspectives d'évolution

Tout ce qui avait été jugé indispensable ou attendu a été livré. La priorisation MoSCoW posée en 4.1 se relit ici, priorité par priorité.

Priorité	Stories	Terminées	Restantes
Must	25	25	—
Should	8	8	—
Could	8	2	6
Won't	5	—	5
Total	46	35	11

Aucune exception chez les must ni chez les should. Les onze stories qui restent sont six could et cinq won't : rien de ce qui avait été engagé ne manque.

Le reste dessine les perspectives d'évolution. Elles attendent dans le backlog, écrites avec leurs critères d'acceptation ; trois autres pistes sont apparues en cours de route et n'y sont pas encore inscrites. Quatre comptent plus que les autres :

- Le dépôt d'offres par les recruteurs eux-mêmes. C'est la plus importante : elle rendrait le site de moins en moins dépendant des interfaces publiques, et lui apporterait des annonces de première main plutôt que recopiées.
- Une adresse relais et un CV anonymisé, pour que le candidat postule sans livrer ses coordonnées avant d'être retenu.
- L'export en PDF des candidatures, comme preuve de recherche d'emploi.
- L'ouverture du catalogue à des sources internationales, derrière un filtre, pour les candidats mobiles ou ouverts au télétravail.

Les dix autres attendent, écrites.

- Des alertes par courriel sur les nouvelles offres correspondant au profil.
- Un badge de relance après un délai sans réponse.
- Des recommandations d'offres, du multi-CV et des statistiques de recherche.
- La stack technique enregistrée dans le profil.
- Un radar des langages les plus recherchés, en France et par région.
- Un motif anonyme de suppression de compte.
- Un rapprochement automatique entre les profils et les offres.
- Une messagerie avec le recruteur.
- Une candidature automatique sur les plateformes externes.
- Un abonnement pour financer l'ensemble.

>_ 15. Épilogue

Le site tourne sans moi. Il interroge trois interfaces publiques sept fois par jour, et il met en avant les offres qu'un débutant peut réellement viser. C'était la promesse du premier jour.

J'ai pris beaucoup de plaisir à réaliser ce projet, même s'il m'a valu quelques nuits à rêver de code. J'en sors grandi, et sûr d'avoir trouvé ma voie. Je compte la poursuivre, en préparant le titre de concepteur développeur d'applications ou en rejoignant une équipe.

Ce site ne restera pas un exercice. J'ai pris un nom de domaine et un hébergement payant chez OVH, là où un projet de formation se dépose d'ordinaire sur une offre gratuite. Le coût est mensuel et la décision est prise : les évolutions de la section précédente ne sont pas une liste de vœux.

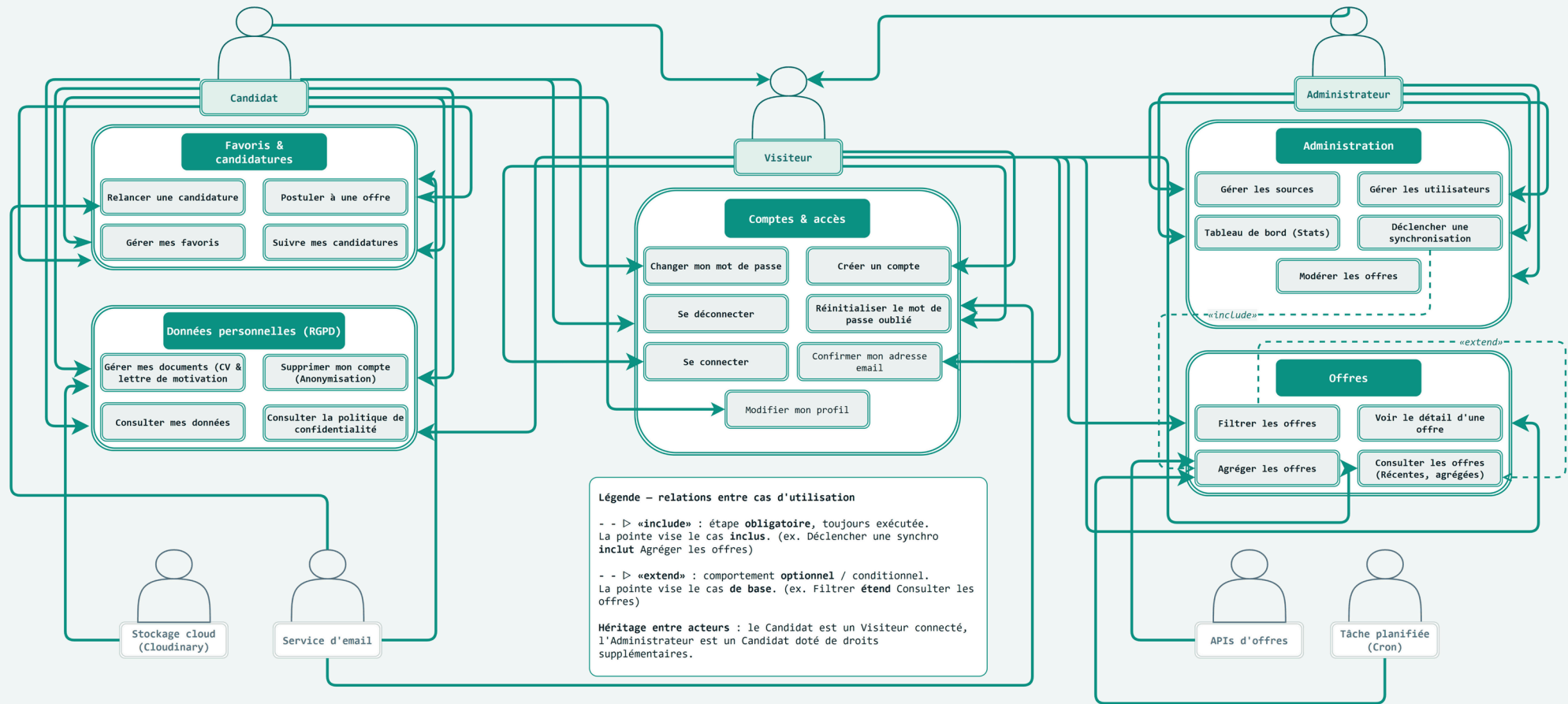
Il ne me reste qu'à m'en servir : le premier candidat à chercher son premier poste avec ce site, ce sera moi.

Annexe 1. User Stories : 6/46

ID	En tant que	Je veux	Afin de	Critères d'acceptation	Priorité	Statut
US01	Visiteur	créer un compte (email + mot de passe)	accéder aux fonctionnalités candidat (profil, favoris, candidatures)	Email unique et valide Politique de complexité du mot de passe (longueur minimale) Mot de passe haché avec bcrypt, jamais stocké en clair Acceptation explicite de la politique de confidentialité (case à cocher non pré-cochée, liée à US07) Message d'erreur si l'email est déjà utilisé Protection CSRF sur le formulaire	Must	Terminé
US02	Candidat	me connecter avec mon email et mon mot de passe	accéder à mon espace personnel	Vérification via bcrypt.compare Création d'une session serveur (cookie) Message générique si identifiants invalides (pas de fuite d'information) Protection CSRF	Must	Terminé
US03	Candidat	me déconnecter	sécuriser mon compte, notamment sur un poste partagé	Destruction de la session Redirection vers l'accueil	Must	Terminé
US04	Visiteur	réinitialiser mon mot de passe oublié	retrouver l'accès à mon compte quand je suis bloqué dehors	Demande à partir de mon email Email envoyé avec un lien à usage unique et à durée limitée (token expirant) Le lien mène à un formulaire de nouveau mot de passe (politique de complexité) Nouveau mot de passe haché avec bcrypt Token invalidé après usage Message générique qu'un compte existe ou non (pas de fuite d'information)	Should	Terminé
US05	Candidat	changer mon mot de passe depuis mon espace	maintenir la sécurité de mon compte	Re-saisie du mot de passe actuel (vérifié via bcrypt.compare) Nouveau mot de passe respectant la politique de complexité Nouveau mot de passe haché avec bcrypt Confirmation + protection CSRF	Should	Terminé
US06	Visiteur	confirmer mon adresse email à partir du lien reçu après mon inscription	activer mon compte et prouver que l'adresse m'appartient	Email envoyé automatiquement après une inscription réussie (Nodemailer/Brevo) Lien à usage unique : le jeton est haché en base, seul le jeton brut circule dans le lien Expiration après 12 h Un seul jeton actif à la fois, les précédents sont invalidés La connexion est refusée tant que l'adresse n'est pas confirmée Message d'inscription identique que l'email existe déjà ou non (anti-énumération)	Must	Terminé

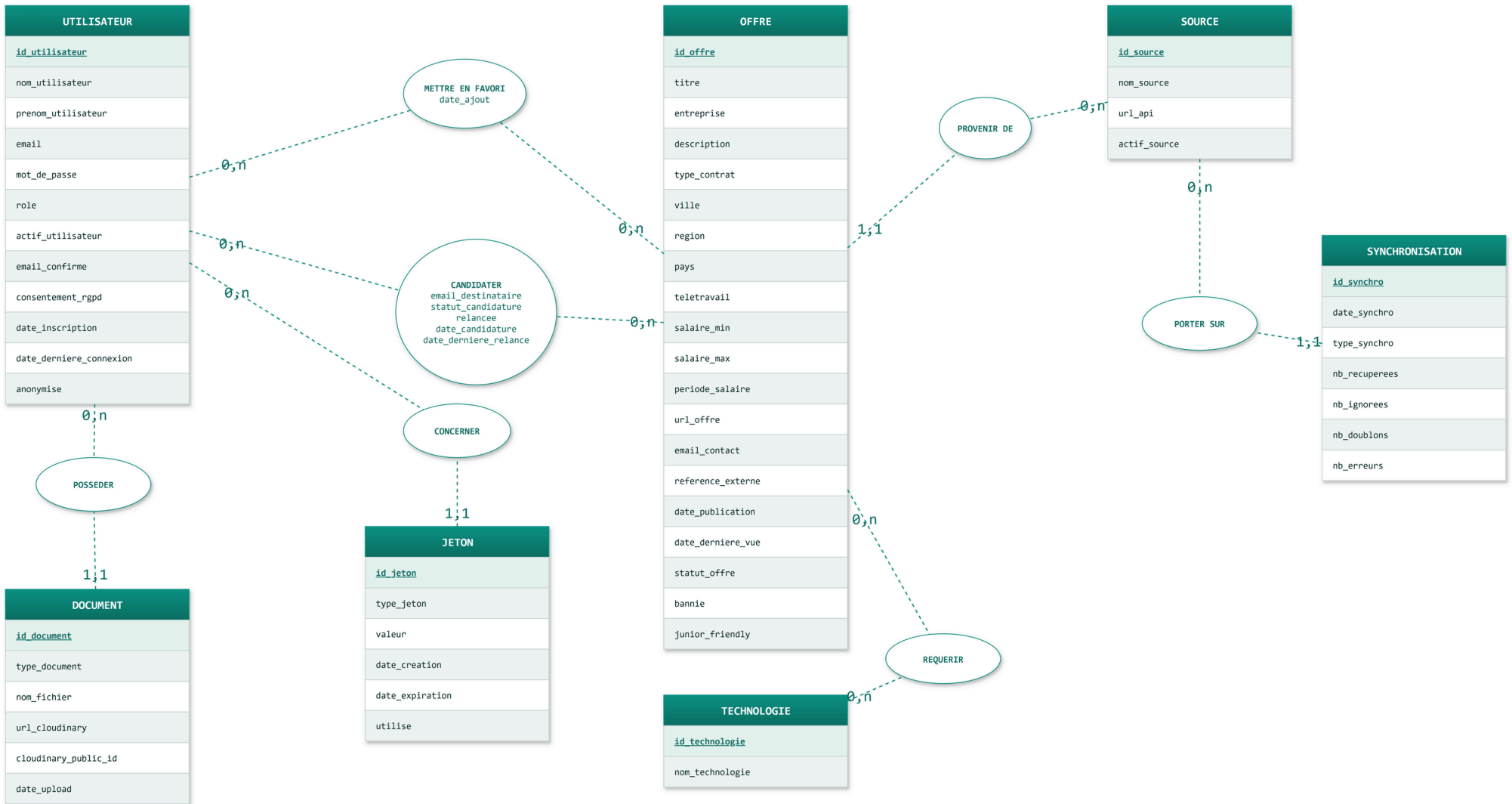
Annexe 2. Le diagramme de cas d'utilisation du MVP

MonPremierPoste - Cas d'utilisation (MVP)



Annexe 3. Le modèle conceptuel de données (MCD)

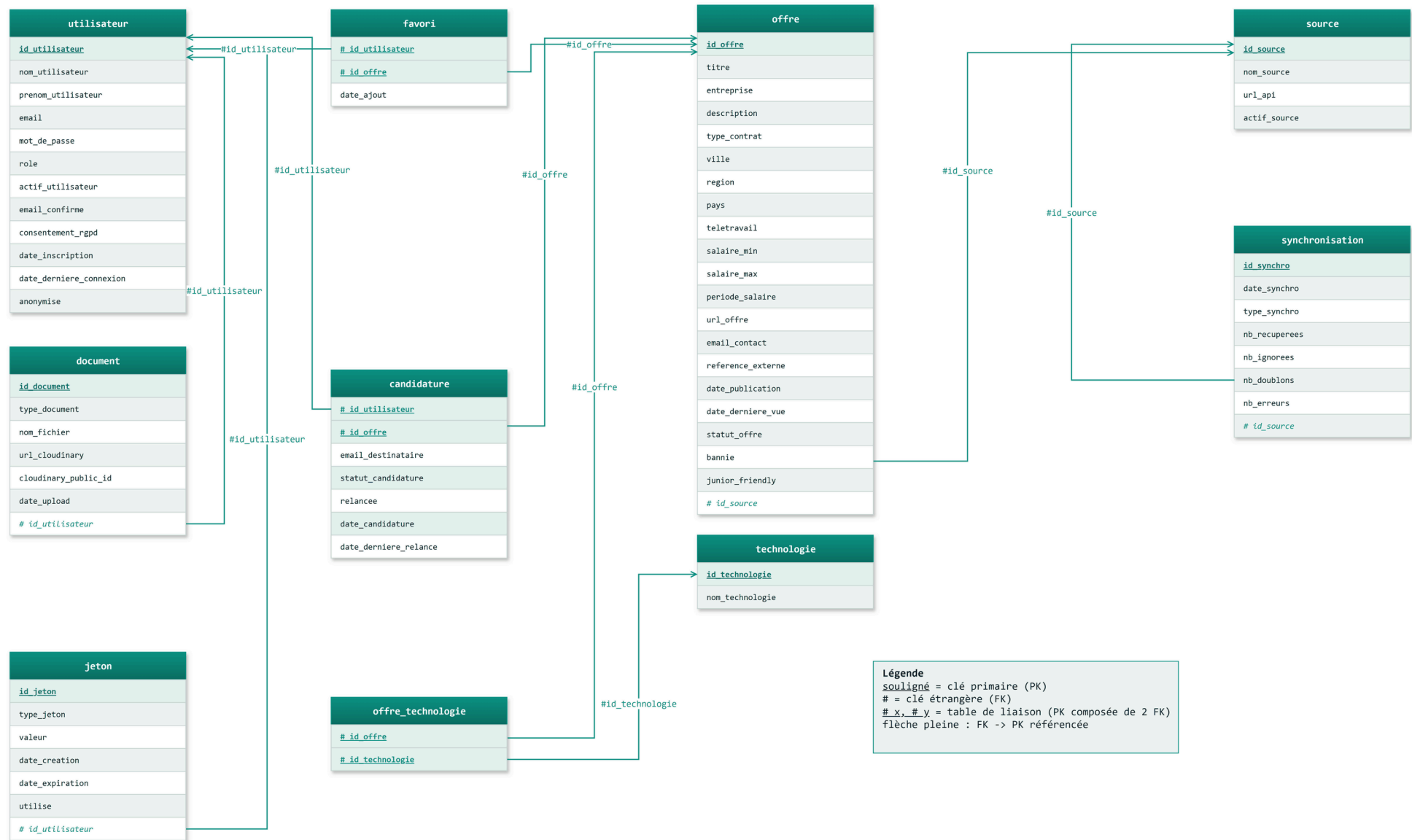
MonPremierPoste – MCD (Modèle Conceptuel de Données)



Notation Merise – cardinalités (min, max) côté entité ; identifiant souligné ; entités (cartes) / associations (verbes).

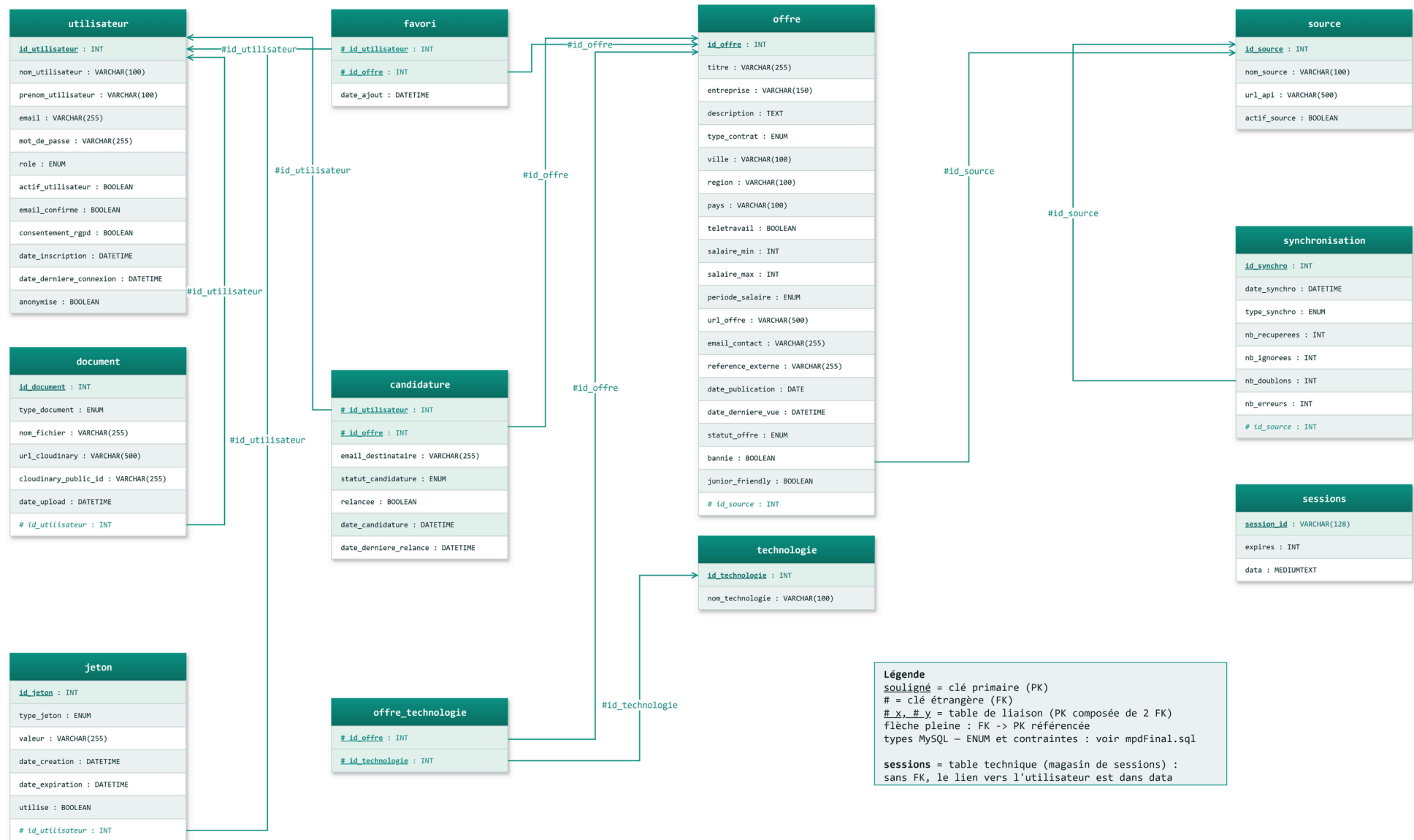
Annexe 4. Le modèle logique de données (MLD)

MonPremierPoste – MLD (Modèle Logique de Données)



Annexe 5. Le modèle physique de données (MPD)

MonPremierPoste – MPD (Modèle Physique de Données)



Annexe 6. Le script de création de la base (1/2 · utilisateur, source, technologie, offre, document)

```
30 -----
31 -- 1. utilisateur
32 -----
33 CREATE TABLE utilisateur (
34   id_utilisateur INT UNSIGNED NOT NULL AUTO_INCREMENT,
35   nom_utilisateur VARCHAR(100) NOT NULL,
36   prenom_utilisateur VARCHAR(100) NOT NULL,
37   email VARCHAR(255) NOT NULL,
38   mot_de_passe VARCHAR(255) NOT NULL,
39   role ENUM('candidat','admin') NOT NULL DEFAULT 'candidat',
40   actif_utilisateur BOOLEAN NOT NULL DEFAULT TRUE,
41   email_confirme BOOLEAN NOT NULL DEFAULT FALSE,
42   consentement_rgpd BOOLEAN NOT NULL DEFAULT FALSE,
43   date_inscription DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
44   date_derniere_connexion DATETIME NULL,
45   anonymise BOOLEAN NOT NULL DEFAULT FALSE,
46   PRIMARY KEY (id_utilisateur),
47   UNIQUE KEY uq_utilisateur_email (email)
48 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
49 -----
50
51 -- 2. source (job APIs: France Travail, Adzuna...)
52 -----
53 CREATE TABLE source (
54   id_source INT UNSIGNED NOT NULL AUTO_INCREMENT,
55   nom_source VARCHAR(100) NOT NULL,
56   url_api VARCHAR(500) NOT NULL,
57   actif_source BOOLEAN NOT NULL DEFAULT TRUE,
58   PRIMARY KEY (id_source),
59   UNIQUE KEY uq_source_nom (nom_source)
60 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
61 -----
62
63 -- 3. technologie (languages and technologies detected in postings)
64 -----
65 CREATE TABLE technologie (
66   id_technologie INT UNSIGNED NOT NULL AUTO_INCREMENT,
67   nom_technologie VARCHAR(100) NOT NULL,
68   PRIMARY KEY (id_technologie),
69   UNIQUE KEY uq_technologie_nom (nom_technologie)
70 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

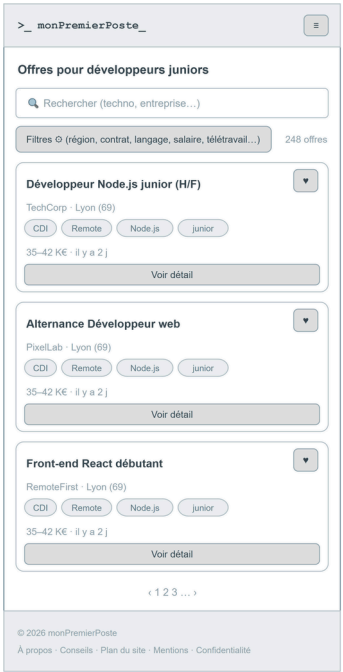
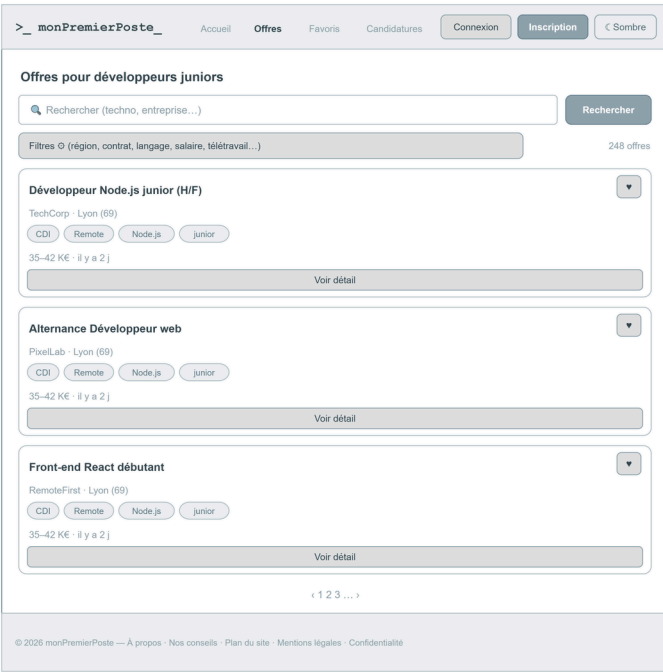
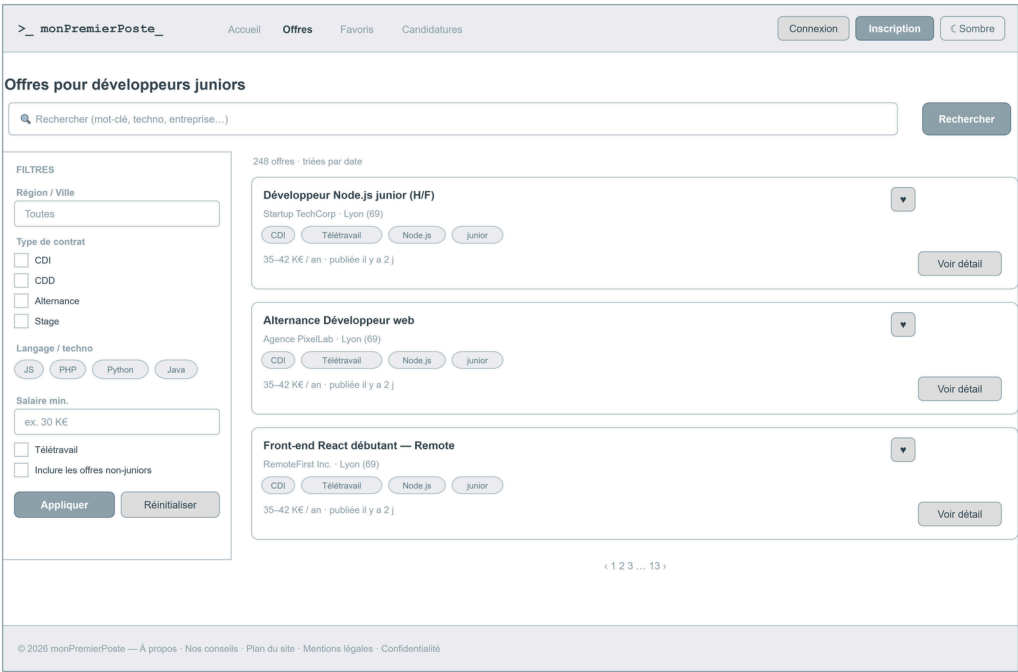
```
75 CREATE TABLE offre (
76   id_offre INT UNSIGNED NOT NULL AUTO_INCREMENT,
77   titre VARCHAR(255) NOT NULL,
78   entreprise VARCHAR(150) NULL,
79   description TEXT NULL,
80   type_contrat ENUM('CDI','CDD','alternance','stage','interim','autre') NULL,
81   ville VARCHAR(100) NULL,
82   region VARCHAR(100) NULL,
83   pays VARCHAR(100) NULL,
84   teletravail BOOLEAN NOT NULL DEFAULT FALSE,
85   salaire_min INT UNSIGNED NULL,
86   salaire_max INT UNSIGNED NULL,
87   periode_salaire ENUM('annuel','mensuel','horaire') NULL,
88   url_offre VARCHAR(500) NULL,
89   email_contact VARCHAR(255) NULL,
90   reference_externe VARCHAR(255) NOT NULL,
91   date_publication DATE NULL,
92   date_derniere_vue DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
93   statut_offre ENUM('active','expiree') NOT NULL DEFAULT 'active',
94   bannie BOOLEAN NOT NULL DEFAULT FALSE,
95   junior_friendly BOOLEAN NOT NULL DEFAULT FALSE,
96   id_source INT UNSIGNED NOT NULL,
97   PRIMARY KEY (id_offre),
98   UNIQUE KEY uq_offre_source_ref (id_source, reference_externe),
99   CONSTRAINT fk_offre_source FOREIGN KEY (id_source)
100   REFERENCES source (id_source) ON DELETE RESTRICT ON UPDATE CASCADE
101 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
102 -----
103
104 -- 5. document (CV / cover letter, FK -> utilisateur)
105 -----
106 CREATE TABLE document (
107   id_document INT UNSIGNED NOT NULL AUTO_INCREMENT,
108   type_document ENUM('cv','lettre') NOT NULL,
109   nom_fichier VARCHAR(255) NOT NULL,
110   url_cloudinary VARCHAR(500) NOT NULL,
111   cloudinary_public_id VARCHAR(255) NOT NULL,
112   date_upload DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
113   id_utilisateur INT UNSIGNED NOT NULL,
114   PRIMARY KEY (id_document),
115   CONSTRAINT fk_document_utilisateur FOREIGN KEY (id_utilisateur)
116   REFERENCES utilisateur (id_utilisateur) ON DELETE CASCADE ON UPDATE CASCADE
117 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Annexe 7. Le script de création de la base (2/2 · jeton, synchronisation, favori, candidature,)offre_technologie, sessions

```
119 -----
120 -- 6. jeton (password reset / email confirmation, FK -> utilisateur)
121 -----
122 CREATE TABLE jeton (
123   id_jeton    INT UNSIGNED NOT NULL AUTO_INCREMENT,
124   type_jeton  ENUM('reset','confirmation') NOT NULL,
125   valeur      VARCHAR(255) NOT NULL,
126   date_creation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127   date_expiration DATETIME NOT NULL,
128   utilise     BOOLEAN NOT NULL DEFAULT FALSE,
129   id_utilisateur INT UNSIGNED NOT NULL,
130   PRIMARY KEY (id_jeton),
131   UNIQUE KEY uq_jeton_valeur (valeur),
132   CONSTRAINT fk_jeton_utilisateur FOREIGN KEY (id_utilisateur)
133   REFERENCES utilisateur (id_utilisateur) ON DELETE CASCADE ON UPDATE CASCADE
134 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
135
136 -----
137 -- 7. synchronisation (sync history, FK -> source)
138 -----
139 CREATE TABLE synchronisation (
140   id_synchro  INT UNSIGNED NOT NULL AUTO_INCREMENT,
141   date_synchro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
142   -- What triggered the sync: scheduled task or admin button.
143   type_synchro ENUM('automatique','manuelle') NOT NULL DEFAULT 'automatique',
144   nb_recuperees INT UNSIGNED NOT NULL DEFAULT 0,
145   nb_ignorees INT UNSIGNED NOT NULL DEFAULT 0,
146   nb_doublons INT UNSIGNED NOT NULL DEFAULT 0,
147   nb_erreurs INT UNSIGNED NOT NULL DEFAULT 0,
148   id_source INT UNSIGNED NOT NULL,
149   PRIMARY KEY (id_synchro),
150   CONSTRAINT fk_synchro_source FOREIGN KEY (id_source)
151   REFERENCES source (id_source) ON DELETE CASCADE ON UPDATE CASCADE
152 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
153
154 -----
155 -- 8. favori (join table utilisateur <-> offre)
156 -----
157 CREATE TABLE favori (
158   id_utilisateur INT UNSIGNED NOT NULL,
159   id_offre INT UNSIGNED NOT NULL,
160   date_ajout DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
161   PRIMARY KEY (id_utilisateur, id_offre),
162   CONSTRAINT fk_favori_utilisateur FOREIGN KEY (id_utilisateur)
163   REFERENCES utilisateur (id_utilisateur) ON DELETE CASCADE ON UPDATE CASCADE,
164   CONSTRAINT fk_favori_offre FOREIGN KEY (id_offre)
165   REFERENCES offre (id_offre) ON DELETE RESTRICT ON UPDATE CASCADE
166 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
168 -----
169 -- 9. candidature (join table utilisateur <-> offre)
170 -----
171 CREATE TABLE candidature (
172   id_utilisateur INT UNSIGNED NOT NULL,
173   id_offre INT UNSIGNED NOT NULL,
174   email_destinataire VARCHAR(255) NOT NULL,
175   statut_candidature ENUM('en_attente','entretien','acceptee','refusee') NOT NULL DEFAULT 'en_attente',
176   relancee BOOLEAN NOT NULL DEFAULT FALSE,
177   date_candidature DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
178   date_derniere_relance DATETIME NULL,
179   PRIMARY KEY (id_utilisateur, id_offre),
180   CONSTRAINT fk_candidature_utilisateur FOREIGN KEY (id_utilisateur)
181   REFERENCES utilisateur (id_utilisateur) ON DELETE CASCADE ON UPDATE CASCADE,
182   CONSTRAINT fk_candidature_offre FOREIGN KEY (id_offre)
183   REFERENCES offre (id_offre) ON DELETE RESTRICT ON UPDATE CASCADE
184 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
185
186 -----
187 -- 10. offre_technologie (join table offre <-> technologie)
188 -----
189 CREATE TABLE offre_technologie (
190   id_offre INT UNSIGNED NOT NULL,
191   id_technologie INT UNSIGNED NOT NULL,
192   PRIMARY KEY (id_offre, id_technologie),
193   CONSTRAINT fk_offtech_offre FOREIGN KEY (id_offre)
194   REFERENCES offre (id_offre) ON DELETE CASCADE ON UPDATE CASCADE,
195   CONSTRAINT fk_offtech_technologie FOREIGN KEY (id_technologie)
196   REFERENCES technologie (id_technologie) ON DELETE CASCADE ON UPDATE CASCADE
197 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
198
199 -----
200 -- sessions - TECHNICAL table, outside the business MCD/MLD/MPD
201 -- (required by express-mysql-session)
202 -----
203 CREATE TABLE sessions (
204   session_id VARCHAR(128) COLLATE utf8mb4_bin NOT NULL,
205   expires INT UNSIGNED NOT NULL,
206   data MEDIUMTEXT COLLATE utf8mb4_bin,
207   PRIMARY KEY (session_id)
208 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Annexe 8. Wireframe de la liste des offres (desktop, tablette et mobile)



Annexe 9. Wireframe de l'administration des sources et des synchronisations (desktop, tablette et mobile)

>_ monPremierPoste_

AccueilOffresFavorisCandidatures

ConnexionInscriptionSombre

ADMINISTRATION

Dashboard

Utilisateurs

Sources & synchro

Offres (modération)

Sources d'offres

Synchro manuelle

Source	État	Dernière synchro	Récupérées	Filtrées	Nouvelles	Doublons	Activer	Actions
France Travail	Active	08/06 06:00	312	118	38	47	▶	Synchroniser
La Bonne Alternance	Active	08/06 06:00	29	0	12	3	▶	Synchroniser
Adzuna	Inactive	05/06 06:00	—	—	—	—	◻	—

20 dernières synchronisations

Date	Source	Type	Récupérées	Filtrées	Nouvelles	Doublons	Erreurs
08/06 06:00	France Travail	Automatique	312	118	38	47	0
08/06 06:00	La Bonne Alternance	Automatique	29	0	12	3	0
07/06 14:22	France Travail	Manuelle	286	102	31	51	2
07/06 06:00	France Travail	Automatique	301	109	24	62	0
07/06 06:00	La Bonne Alternance	Automatique	31	2	8	5	0
06/06 06:00	France Travail	Automatique	295	121	19	58	1
06/06 06:00	La Bonne Alternance	Automatique	28	1	6	4	0
05/06 06:00	Adzuna	Automatique	144	63	11	27	0
05/06 06:00	France Travail	Automatique	288	115	22	49	0
05/06 06:00	La Bonne Alternance	Automatique	27	0	5	6	0
04/06 09:41	France Travail	Manuelle	279	104	17	55	0
04/06 06:00	France Travail	Automatique	284	111	20	53	0
04/06 06:00	La Bonne Alternance	Automatique	30	1	7	4	0
03/06 06:00	France Travail	Automatique	276	98	15	60	1

Compteurs agrégés seulement — le détail d'une source (configuration, quotas, historique) s'ouvre en cliquant son nom.

© 2026 monPremierPoste — À propos · Nos conseils · Plan du site · Mentions légales · Confidentialité

>_ monPremierPoste_

AccueilOffresFavorisCandidatures

ConnexionInscriptionSombre

ADMINISTRATION

Dashboard

Utilisateurs

Sources & synchro

Offres (modération)

Sources d'offres

Synchro manuelle

Source	État	Dernière	Nouv.	Doubl.	Actif	Action
France Travail	Active	08/06 06:00	38	47	▶	Sync.
La Bonne Alternance	Active	08/06 06:00	12	3	▶	Sync.
Adzuna	Inactive	05/06 06:00	—	—	◻	—

20 dernières synchronisations

Date	Source	Type	Récup.	Filtr.	Doubl.	Err.
08/06 06:00	France Travail	Automatique	312	118	47	0
08/06 06:00	La Bonne Alternance	Automatique	29	0	3	0
07/06 14:22	France Travail	Manuelle	286	102	51	2
07/06 06:00	France Travail	Automatique	301	109	62	0
07/06 06:00	La Bonne Alternance	Automatique	31	2	5	0
06/06 06:00	France Travail	Automatique	295	121	58	1
06/06 06:00	La Bonne Alternance	Automatique	28	1	4	0
05/06 06:00	Adzuna	Automatique	144	63	27	0

Compteurs agrégés seulement — le détail d'une source s'ouvre en cliquant son nom.

© 2026 monPremierPoste — À propos · Nos conseils · Plan du site · Mentions légales · Confidentialité

>_ monPremierPoste_

DashboardUtilisateursSourcesOffres

Sources d'offres

Synchro manuelle

France TravailActive08/06 06:00
Récup. 312 · Filtr. 118 · Nouv. 38 · Doubl. 47
Synchroniser

La Bonne AlternanceActive08/06 06:00
Récup. 29 · Filtr. 0 · Nouv. 12 · Doubl. 3
Synchroniser

AdzunaInactive05/06 06:00
Récup. — · Filtr. — · Nouv. — · Doubl. —

20 dernières synchronisations

08/06 06:00France Travail
Récup. 312 · Filtr. 118 · Nouv. 38 · Doubl. 47 · Err. 0
Auto.

08/06 06:00La Bonne Alternance
Récup. 29 · Filtr. 0 · Nouv. 12 · Doubl. 3 · Err. 0
Auto.

07/06 14:22France Travail
Récup. 286 · Filtr. 102 · Nouv. 31 · Doubl. 51 · Err. 2
Manu.

07/06 06:00France Travail
Récup. 301 · Filtr. 109 · Nouv. 24 · Doubl. 62 · Err. 0
Auto.

07/06 06:00La Bonne Alternance
Récup. 31 · Filtr. 2 · Nouv. 8 · Doubl. 5 · Err. 0
Auto.

06/06 06:00France Travail
Récup. 295 · Filtr. 121 · Nouv. 19 · Doubl. 58 · Err. 1
Auto.

06/06 06:00La Bonne Alternance
Récup. 28 · Filtr. 1 · Nouv. 6 · Doubl. 4 · Err. 0
Auto.

05/06 06:00Adzuna
Récup. 144 · Filtr. 63 · Nouv. 11 · Doubl. 27 · Err. 0
Auto.

Compteurs agrégés seulement.

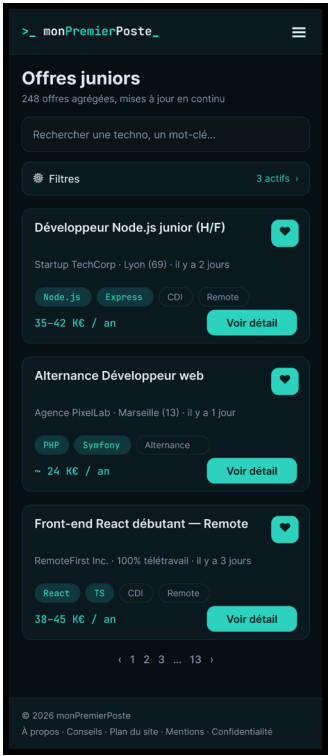
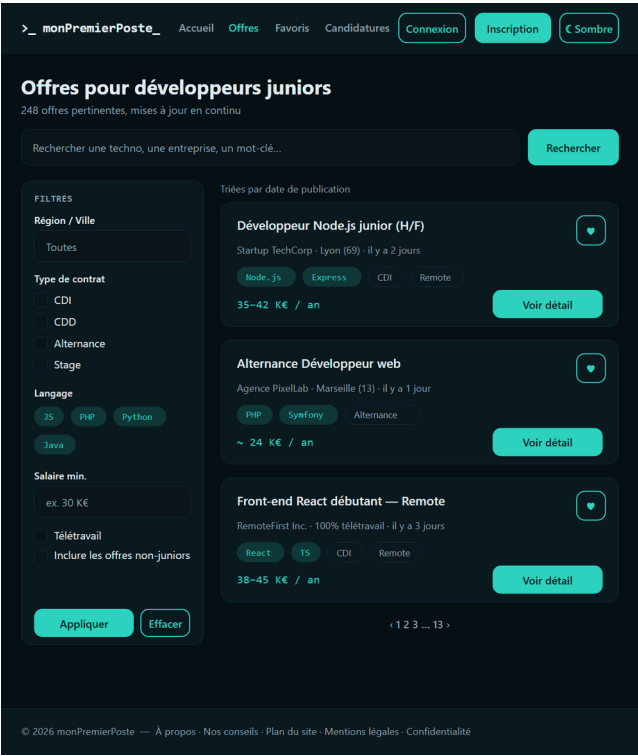
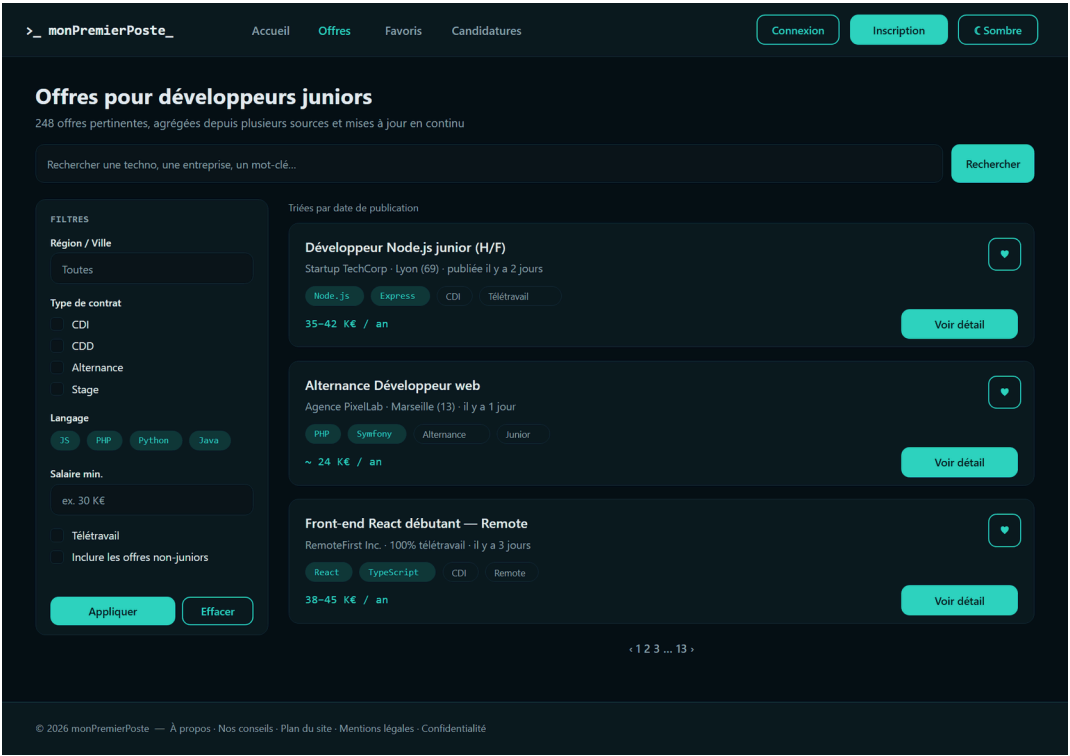
© 2026 monPremierPoste
À propos · Conseils · Plan du site · Mentions · Confidentialité

sommaire

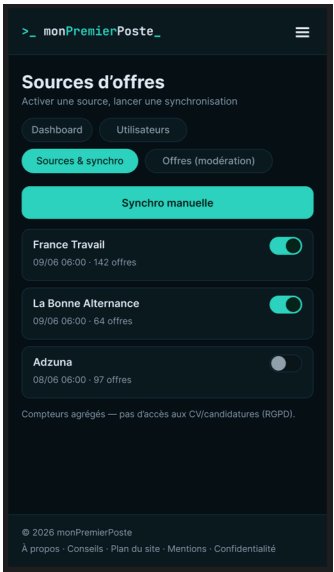
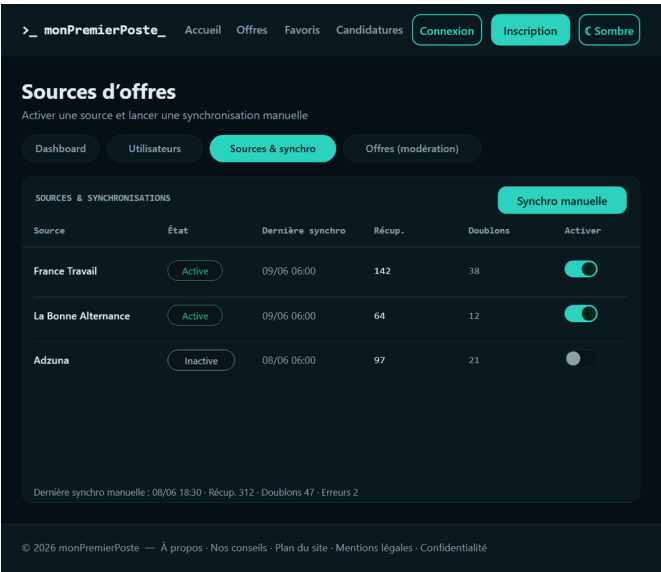
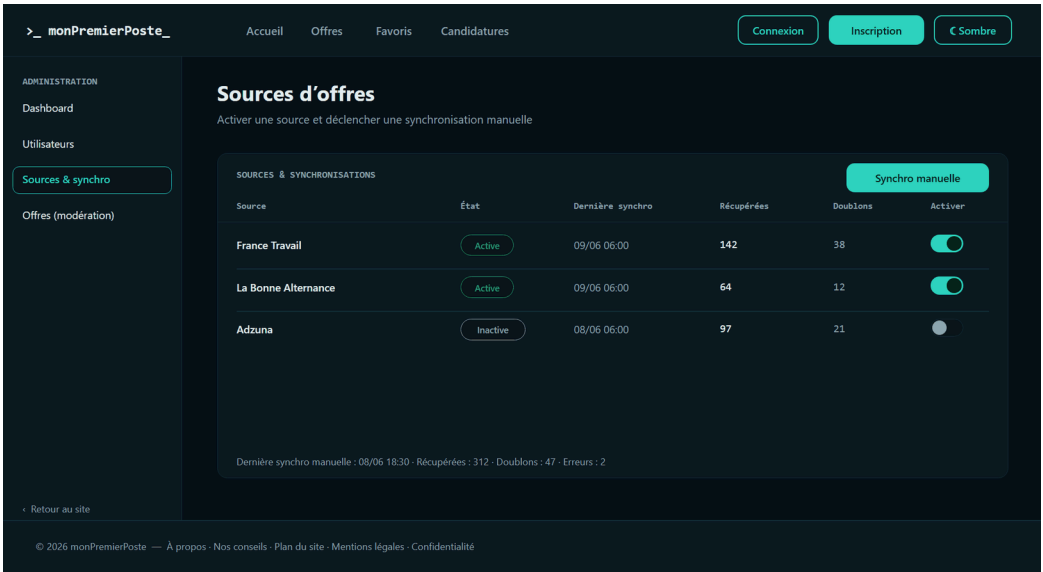
59

annexes

Annexe 10. Maquette de la liste des offres (desktop, tablette et mobile)



Annexe 11. Maquette de l'administration des sources et des synchronisations (desktop, tablette et mobile)



Annexe 12. Le système de composants de la maquette (7 des 13 composants)

Système de composants

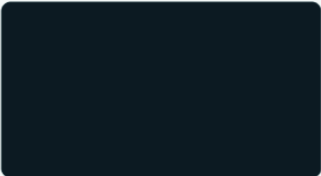
MonPremierPoste · bibliothèque de la maquette bureau · 13 composants, 56 variantes, 33 styles

01 Base

Les briques. Elles ne connaissent pas la page qui les emploie.

Panneau

Le motif le plus répété du site après l'en-tête et le pied : dix-huit emplois. Il accueille le panneau de filtres, les cartes de formulaire, les blocs de contenu et les tableaux. Sa taille est décidée par ce qu'il reçoit, jamais par lui-même.



Bouton

Vingt emplois dans les écrans. Principal pour l'action attendue de la page, Secondaire pour les autres. La largeur suit le libellé par mise en page automatique.



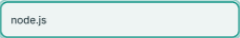
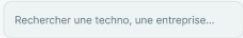

Puce

Techno pour les langages et les cadriciels, en police à chasse fixe et sur un fond d'accent transparent ; Méta pour le contrat et le lieu, en simple contour. La largeur suit le libellé.




Champ

Dix-neuf emplois. L'état Focus n'est pas un ornement : il épaissit le contour, ajoute un halo et fait passer le texte de la couleur atténuée à la couleur pleine. C'est ce qui rend le formulaire suivable au clavier.

Case à cocher

Les filtres de contrat et de télétravail. Cochée, elle se remplit de l'accent et porte la coche dans la couleur du texte sur accent : le contraste est tenu dans les deux thèmes.




Badge

Les écrans d'administration. Positif et Négatif portent le vert et le rouge qui ont désormais un nom ; Accent marque le rôle administrateur, Neutre est l'état de repos.




02 Blocs

Ils assemblent des briques et portent un contenu.

Carte d'offre

La liste des offres et les favoris. Elle réunit une carte, deux puces techno, deux puces méta et un bouton principal. Elle ne change pas d'état : elle change de contenu, par remplacement de texte dans l'instance.

Développeur Node.js junior (H/F)

Startup TechCorp - Lyon (69) · publiée il y a 2 jours



35-42 K€ / an



Développeur Node.js junior (H/F)

Startup TechCorp - Lyon (69) · publiée il y a 2 jours



35-42 K€ / an



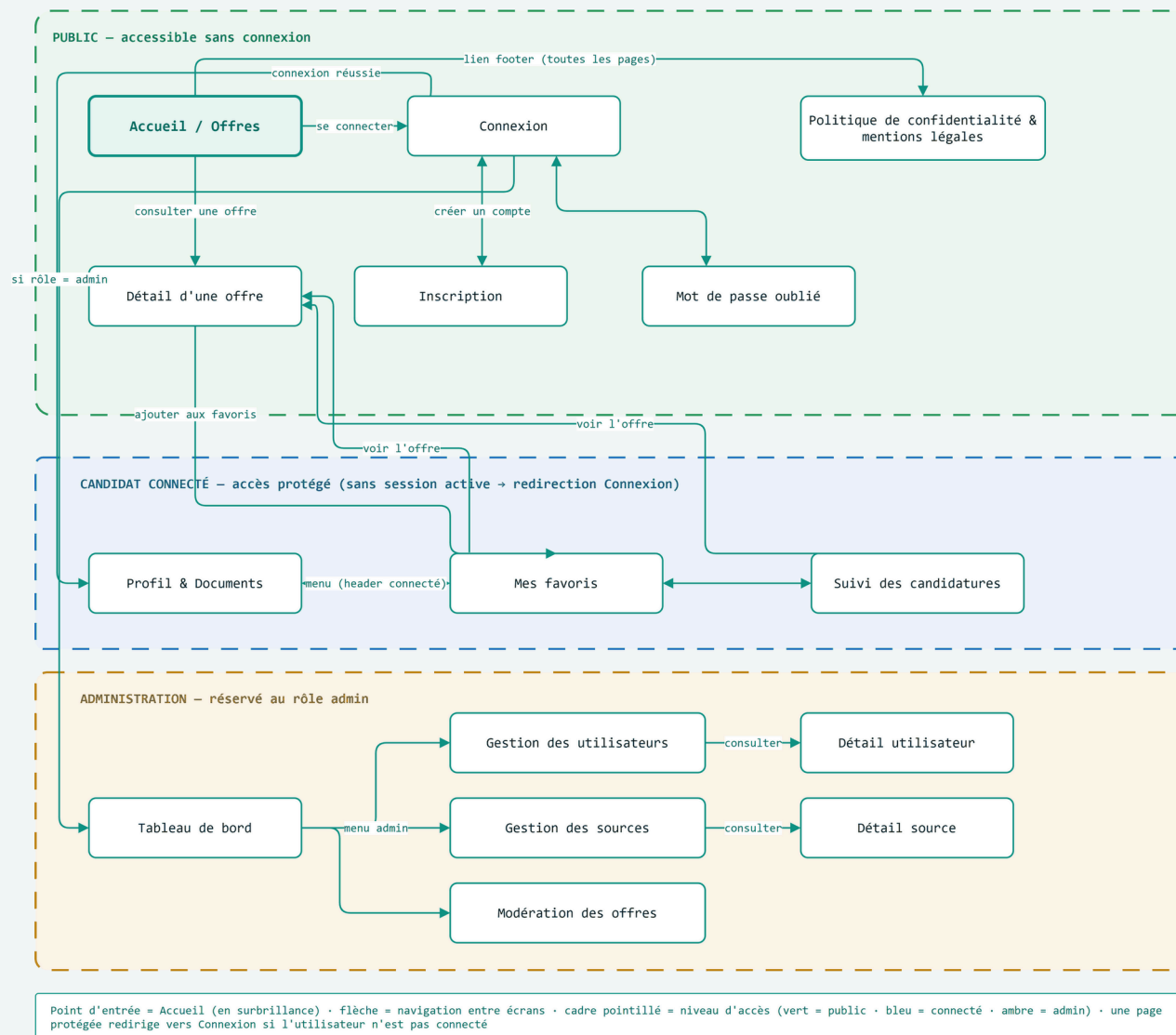
sommaire

62

annexes

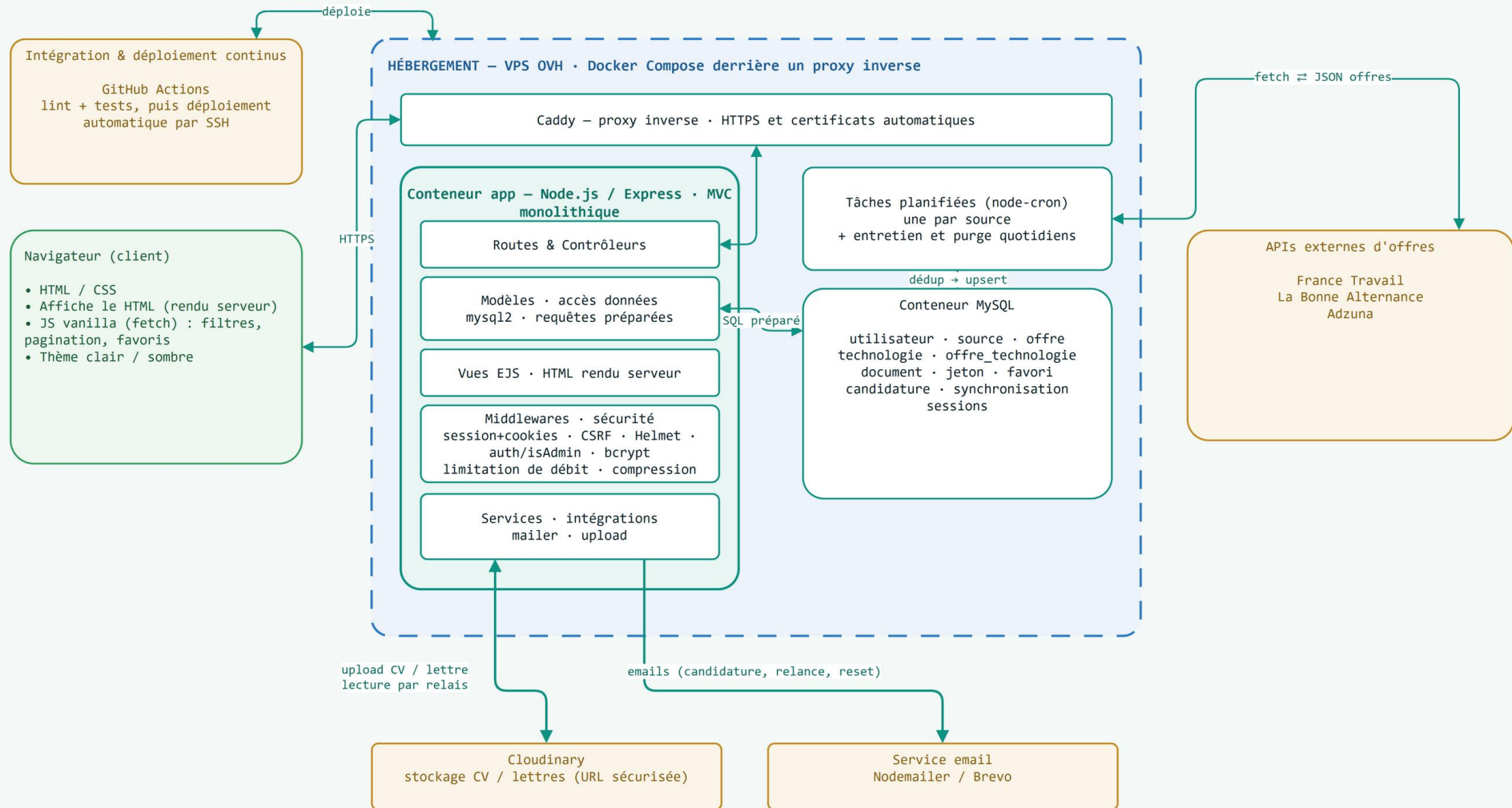
Annexe 13. Le diagramme de navigation (les trois niveaux d'accès)

MonPremierPoste – Diagramme de navigation



Annexe 14. Le schéma d'architecture (hébergement, services tiers et flux)

MonPremierPoste – Schéma d'architecture



Vert = client · teal = composants applicatifs · bleu = périmètre d'hébergement · ambre = services tiers · flèche = échange de données (HTTPS / SQL)

Annexe 15. Le MVC dans l'arborescence du dépôt

MonPremierPoste – Le MVC dans l'arborescence du dépôt

```
mon-premier-poste/
├── .github/
│   ├── workflows/
│   └── public/
├── css/
├── js/
├── images/
├── fonts/
├── robots.txt
├── sitemap.xml
├── scripts/
├── sql/
├── src/
│   ├── config/
│   ├── models/
│   ├── controllers/
│   ├── routes/
│   ├── middlewares/
│   ├── validators/
│   ├── services/
│   ├── utils/
│   ├── views/
│   ├── offres/
│   ├── favoris/
│   ├── candidatures/
│   ├── admin/
│   ├── errors/
│   └── partials/
├── app.js
├── tests/
│   ├── unit/
│   └── integration/
├── server.js
├── Dockerfile
├── docker-compose.yml
├── docker-compose.override.yml
├── docker-compose.prod.yml
├── .dockerignore
├── Caddyfile
├── package.json
├── package-lock.json
├── eslint.config.js
├── prettier.config.js
├── .env.example
├── .gitignore
├── .gitattributes
├── .prettierrignore
└── README.md

# racine du dépôt
# intégration et déploiement continu
# ci.yml : lint, tests, puis mise en ligne
# fichiers statiques servis au navigateur
# feuilles de style (charte graphique)
# JS vanilla : filtres fetch, favoris, thème
# favicon, image Open Graph, logo Adzuna (licence)
# polices auto-hébergées (woff2)
# zones privées interdites aux robots, adresse du plan
# pages à indexer (les offres en sont exclues)
# exploitation : sauvegarde, déploiement, jeu d'essai
# schema.sql (structure) · privileges.sql (droits)
# code serveur – application MVC
# base, session, courriel, sources, tâches planifiées, règles métier
# M · accès données (mysql2, requêtes préparées)
# C · lit la requête, appelle, choisit la vue
# définition des routes Express
# sécurité : auth, CSRF, limitation de débit
# validation & nettoyage des entrées
# logique métier + intégrations (courriel, upload)
# fonctions utilitaires (formatage)
# V · templates EJS (HTML rendu serveur)
# liste et détail des offres
# offres mises de côté
# suivi et relances
# espace d'administration
# pages 403, 404, 429, 500
# en-têtes, pieds, composants réutilisables
# construit & exporte l'app Express
# suite Jest : unitaires et intégration
# fonctions isolées
# requêtes HTTP réelles (Supertest, base dédiée)
# point d'entrée : base, tâches planifiées, démarrage
# image de l'application (node:22-slim)
# orchestration application + base de données
# surcharge locale, chargée automatiquement
# surcharge production : ajoute le proxy inverse
# fichiers exclus de l'image
# proxy inverse : HTTPS et certificats
# dépendances & scripts npm
# versions verrouillées des dépendances
# qualité de code (ESLint)
# formatage du code (Prettier)
# modèle de variables d'environnement
# fichiers exclus du suivi Git
# normalisation des fins de ligne (LF)
# fichiers ignorés par Prettier
# présentation & installation du projet
```

M = Modèle (accès aux données) · V = Vue (présentation EJS) · C = Contrôleur (orchestration) – les trois couches du MVC